Trials@uspto.gov
571-272-7822

Exhibit No. 2043

UNITED STATES PATENT AND TRADEMARK OFFICE

BEFORE THE PATENT TRIAL AND APPEAL BOARD

MICROSOFT CORP.,
Petitioner,

v.

ROYAL J. O'BRIEN,
Patent Owner.

U.S. Patent No. 8,380,808
Issued: Feb. 19, 2013
Filed: Nov. 24, 2008

Patent Title: DYNAMIC MEDIUM CONTENT STREAMING SYSTEM

Inter Partes Review No. 2021-00015

DECLARATION OF WILLIAM ANTHONY MASON

Contents

I. Introduction

This declaration is a supplement to the declaration that I previously provided in this case (Ex. Paper 7), and I hereby incorporate the information set forth in that document by reference.

I have prepared this declaration to address specific issues raised in the Patent and Trial Appeal Board's decision of April 28, 2021, in which the board made an initial determination that the teachings of the '808 Patent appear to be sufficiently shown to be obvious to warrant institution in light of the prior art for all but claim 13 of the Patent. I maintain that the prior art, if properly understood, cannot render the claims of the '808 Patent obvious. Thus, I suggest that the differences between the referenced prior art and the '808 Patent might have been misunderstood because:

1. The '808 Patent describes a system that is driven by the client computer. The '808 Patent limitations do not include a server but the specifications of the cited prior art teach that a server is required. No combination of the prior art references presented by the Petitioner can be read to teach the client-only implementation that is claimed by the '808 Patent. Claim 1 of the '808 Patent includes the limitation that the system is "[a] streaming on demand system comprising an on-demand requester object installed on a computing device..." (Ex. 1001, Claim 1). No other computing device is required by the limitations of independent Claim 1. Similarly, Claim 14 of the '808 Patent includes the limitation that "[a] streaming on demand method comprising installing an on-demand requester object on a computing device having an I/O manager..." Again, no other computing device is required by the limitations of independent Claim 14. The specification of Eylon teaches that a streaming on-demand system includes a server: "Turning to FIG. 1, there is shown block diagram of a system implementing various aspects of the present invention. The system includes a server 12 which is connected to one or more clients 14 via a data network 16, such as the Internet, an intranet, extranet or other TCP/IP based communication network, or other types of data networks, including wireless data networks." (Ex. 1005, Col. 5, Lines 45-52). Similarly, the specification of Christiansen teaches that cached objects reside on a remote

server: "Briefly, the present invention provides a method and system for caching remote objects locally. A request to access an object is received. A determination is made as to whether the object is cached. If the object is cached and the request is not to create a new object, modify an existing object, or open a directory, the request is directed to a local file system. Otherwise, the request is directed to a remote file system." (Ex. 1006, Para. 4). The combination of Eylon and Christiansen do not teach a system that lacks a server. Neither Eylon nor Christiansen nor any combination of the two proposed by the Petitioner can provide a workable system without a server. The system taught by the '808 patent can use a server, but the '808 patent teaches and claims a system that works without such a server. By itself, this distinction is sufficient to demonstrate a difference between the claims of the '808 Patent and the teachings of the cited prior art.

2. The '808 Patent teaches the difference between application I/O and paging I/O: "If the entire data pack has not been received, as determined in step 418, a determination is made if the call is a paging I/O or a normal read. Paging (sometimes called swapping) is the transfer of pages between main memory and an auxiliary store, such as hard disk drive, which allows use of disk storage for data that does not fit into physical RAM. In a paging I/O, compressed packs are unpacked into allocated paging memory buffers at missing offsets in step 428, and control proceeds to step 432. In a normal read I/O, a new read request is created and sent for the same allocation to a lower filter driver and the original buffer is filled, as in step 438. Again, control is then passed to step 432." (Ex. 1001, Col 7, Lines 48-60). To reinforce this, I have provided extrinsic evidence that application I/O and paging I/O are different (§ I.). Modern operating systems rely upon paging I/O to implement file system data caching. However, paging I/O is only initiated by the operating system, not by the application program. Note that Claim 1 explicitly states: "... said on-demand requester object being responsive to application I/O requests, said application I/O requests corresponding to sequential and non-sequential data packs, said data packs being portions of a complete data stream, said I/O requests determining an order in which data packs are

sought for the application..." (Ex 1001Claim 1). Paging I/O is distinct from application I/O and the claim is specific to being responsive to application I/O requests. A system that is only responsive to operating system I/O requests (which includes all paging I/O requests) would not meet the limitations of the claims language. The same language is also present in independent Claim 14. Accordingly, the distinction between application I/O and paging I/O is core to the claims of the '808 Patent. Neither Eylon nor Christiansen teach this distinction.

3. The '808 Patent teaches that it does not rely upon a redirector or a virtual file: "An important aspect of a process according to principles of the invention is that it does not require any virtual drive, device or the like. It works with the original hard drive and disk partition. Many application streaming systems and processes that currently exist facilitate streaming by using a virtual drive, file system driver or the like which are difficult to implement in compliance with security protocols and inefficient to maintain. In contrast, the subject invention uniquely uses a mini-filter, also known as a file system filter driver, i.e., an object that attaches to a file system driver and filters requests only. It does not generate or originate requests like a file system driver does. Consequently, in implementations of the subject invention, data is provided directly from the operating system hard drive partition, not a virtual file, redirector or the like." (Ex. 1001, Col 10 Line 62 through Col 11 Line 8.) Petitioner argues that the combination of Eylon and Christiansen is obvious but does not describe to us if this combined system continues to provide the virtual drive model of Eylon: "When a streaming application is launched, it is configured to indicate the VFS 160 as the source for application files." (Ex. 1005, Col. 8 Lines 27-29). Christiansen teaches the use of a redirector (Ex. 1006Figure 7, Figure 11, para 30, 55, 56, 60, 62, 64, 72, 79, 81). The Petitioner asserts that the combination of Eylon and Christiansen makes the claims of the '808 Patent obvious, but does not adequately explain how systems relying on virtual files and redirectors combine to form the system of the '808 Patent that requires neither of these. It is my opinion that they do not offer such an explanation.

4. The definition of a Person of Ordinary Skill in the Art ("POSITA"), proposed by the Petitioner would not have been motivated nor capable of adapting Eylon with reference to Christensen to arrive at the claimed invention of the '808 Patent. This inability to produce the claimed invention means that the POSITA would not find the invention obvious. Assuming the POSITA finds the unusual combination of prior art proposed by the Petitioner gives a false view of the claims. This false view of the claims leads to incorrect conclusions. I will explain how I understand the Petitioner's definition of a POSITA and demonstrate that such a person would not have combined the cited prior art references in the fashion described. Even the Petitioner's own proposed novel combination of the prior art references does not lead to a system that teaches the claims of the '808 Patent, despite the remarkable knowledge given to the POSITA using the Petitioner's definition.

5. In my prior declaration, I was not sufficiently clear as to why the offered motivation to combine the prior art references is incorrect. In this declaration I provide additional explanation as to why a POSITA would not have been motivated to combine the prior art references in the novel manner proposed by the Petitioner, both because the Petitioner's combination is counter to what is taught in the very prior art the Petitioner seeks to combine, and why the Petitioner's proposed combination, even if deemed obvious, would not lead to a workable solution without compromising the teachings of the prior art or raising the level of complexity far beyond what is represented by the Petitioner, despite Petitioner's own experience implementing systems like the Petitioner's proposed combination. I am also qualified to explain how, in my own experience constructing systems like that proposed by Petitioner, using file systems, file system filters, and file system mini-filters, such a system is much more complex than the systems taught by the prior art.

6. The '808 Patent describes a dual mapping layer, which is essential to understanding the teachings and claims of the '808 Patent. I provide additional analysis regarding this teaching and the resulting claim limitation of the '808 Patent and then observe specific features/benefits

of that teaching which are not described by any of the cited prior art works, as well I will go on to demonstrate why specific elements of the claims, as supported by the specification, are required to reside on the client computer system, do not depend upon any server functionality, and that such a limitation is absent from the prior art.

In presenting my opinions, I have also cited evidence such as textbooks and articles, to support those opinions. I have added an additional section (§III.) in which I provide a more thorough explanation of how I reached my conclusions; I have removed it from the main analysis because it involves considerable technical detail. Given that I have more than three decades of experience building operating systems and file systems, I have considerable depth of understanding. In my analysis, I have focused on information that would have been generally known to a POSITA using the Petitioner's proposed definition.

II. Patentability Analysis

A. Level of Ordinary Skill in the Art

I use the term POSITA to mean a Person of Ordinary Skill In The Art. The Petitioner, Microsoft, has used the term "Skilled Artisan". I consider the two terms to be equivalent.

In my opinion the person that the Petitioner has constructed as a POSITA is in fact a unique expert with remarkable skills of prescience and hindsight as this simplifies their analysis that a POSITA finds the teachings of the '808 Patent obvious over the cited prior art references.

I have reached my opinion — that the Petitioner's POSITA has a remarkable suite of skills and abilities — because, based upon the analysis the Petitioner attributes to this POSITA, it is clear the Petitioner's POSITA not only has a broad range of knowledge across rare and even "arcane" areas, the POSITA also demonstrates a depth of understanding about the nuances of operating systems that I have spent much of my own career learning and contributing. I have been advised by counsel that the law does not permit such a specialized expert serving as a POSITA.

The Petitioner's definition seems, on its face, to be reasonable: someone with a bachelor's degree and four years of experience or a master's degree and two years of experience. However, the breadth and depth of knowledge attributed to the Petitioner's POSITA is astonishing. It requires an array of specific experiences, including computer networking, multiple operating systems, file systems (including physical media file systems, network file systems, and virtual file systems), filter drivers, and file system minifilters, encryption technology, compression technology, and user mode and kernel mode programming. In my experience this breadth and depth of knowledge and understanding is gained either by someone with extraordinary skills — usually during their PhD studies — or by someone with many years of experience.

The breadth and depth of knowledge attributed to the Petitioner's POSITA made me question whether I had the requisite understanding across the swath of domains that the Petitioner's POSITA had. I have convinced my self that I do, however, I note that in 2007 I had 20 years of post-bachelor's experience and I was still trying to solve some of the problems that arise in

the creation of the minifilter that Dr. Houh proposes. Thus, it is possible that I would not have qualified as a POSITA using the Petitioner's definition.

The Petitioner's POSITA is versed in a variety of areas of computer science:

- Virtual File System — this term, as used by Eylon — "... a sparsely populated virtual file system ('VFS') ..." (Ex. 1005, Col 4, Lines 3-4), is somewhat different than the ordinary use of this term in 1998, the priority date of Eylon's application. The only book available on Windows file systems development was "Windows NT File Systems Internals: A Developers Guide" (Ex. 2017) and the term "VFS" does not appear in its text. The UNIX operating system's definition of a virtual file system (VFS) is quite broad because any file system that wished to interface with the operating system was a virtual file system. For example, NTFS running on Linux today is implemented as a Linux VFS. I would note, however, that a POSITA under the Petitioner's expansive definition would not have known what the term VFS meant because file systems are not part of the standard undergraduate or graduate computer science curriculum in 1998 or 2008. Even students that had studied the optional operating systems courses in this time would not have been exposed to this terminology. I discuss this further in § A.. Similarly, someone with a Master's degree in Computer Science might have learned about UNIX VFS, though it was certainly not a core requirement of graduate CS education at the time and thus does not fall within the Petitioner's own expansive definition of a POSITA. I discuss this further in § B.. I taught seminars about Windows file systems developments between 1996 and 2016 and I do not know anyone else in the world that was offering similar in-depth training, either as part of an undergraduate or graduate education program in computer science or any other discipline or even as a commercial seminar series, such as the one that I taught (Ex. 2044, and 2045, both from 1997). There were textbooks about operating systems available, but many did not cover file systems; those that did would ordinarily teach the VFS interface used by the UNIX operating systems. This is because the source code for UNIX and Linux was freely available; the Petitioner has never had a freely available version of the Windows operating

system source code available. I discuss operating system textbooks and their coverage of file systems further in § C..

- File System Filter — this is a specific feature of Windows that permits a software driver — the "filter driver" that can be used to provide additional processing or functionality on top of a file system. These have existed since Windows NT 3.1 (1993) and were described in Nagar's book on Windows File Systems Development in 1997, which is prior to the time of any of the cited prior art. (Ex. 2017) No other operating system provides a file system filter. While it is like the layered/stackable file systems described by Zadok (Ex. 1014), such discussion was not covered by the standard undergraduate curriculum, any graduate curriculum from that period I could find, or any of the file system textbooks available in 2008 or even more recently. I discuss undergraduate curriculums in 2008 in § A.. I discuss graduate curricula in § B.. I discuss textbooks from that time to the current time in § C..

- Installable File System — this is a specific interface of Microsoft Windows. The Petitioner had a public conference on building installable file systems in 1994. In 1997 Nagar published a book about building file systems for Windows (Ex. 2017). Constructing an installable file system does not use the same interface as the VFS interface of the UNIX system. I know this because: (1) I designed, developed, and implemented a file system toolkit for Windows that allowed porting of VFS-based file systems on UNIX to Windows in the mid-1990s; this was a substantial amount of work to do, but it simplified the porting of file systems from UNIX platforms to Windows platforms. I describe this further in § D.; and (2) I was in the business of assisting people in building new file systems for Windows and porting existing file systems to Windows from 1994 to 2016. The IFS FAQ of which I was the primary author, last updated in 2007, explicitly states: "Q20 Does NT/2K/XP have anything like a VFS interface? The Virtual File System (VFS) interface is a UNIX abstraction originally developed in order to simplify the porting of the Network File System

(NFS) to various UNIX platforms. It provides a model where there is a 'standard' set of operations that are supported by a file system. The operating system then, in turn, works with the file system to maintain a 'vnode' cache (where a 'vnode' is a virtual file system information element) and to provide core OS management services for working with the OS. The Windows NT/2000/XP platform also has a "standard" set of operations that it expects to be supported by file systems. The operating system then invokes the relevant file system services by calling upon those standard services. However, the details of the implementation of the UNIX VFS interface are quite different than the IRP Major dispatch entry points a Windows NT file system. Because of this, a UNIX VFS-based file system does not 'port' in any trivial fashion to Windows NT/2000/XP." (Ex. 2048)

- Streaming Delivery — the term "streaming" is used by Eylon, Vinson, and the '808 Patent. Eylon uses it in the abstract and throughout the patent (Ex. 1005). Vinson's patent is assigned to a company called "Stream Theory" and the terminology is used in materials associated with Vinson (Ex. 2067). The '808 uses it in its title ("DYNAMIC MEDIUM CONTENT STREAMING SYSTEM") and throughout its specification and claims (Ex. 1001). Thus, during the time under consideration the term "streaming" would typically refer to internet technologies for moving data from one computer to another in a form that permitted mostly error-free real-time use of the data. Often this was done inside the web browser. These technologies would ordinarily not have involved any kernel level programming or need for someone building a data streaming service to know anything about file systems, file system filter drivers, virtual file systems, installable file systems, or other operating systems concepts. I discuss streaming technologies in § F.. Streaming delivery, or data streaming might have been included as part of basic network education at both undergraduate and graduate level. I discuss undergraduate computer science education requirements in §subsection A.. This was not a topic that was discussed as part of operating systems education, however, as can be seen from the OS textbooks. I discuss operating system textbooks in § C.

- Demand Paging — this is a term ordinarily used to describe the loading of data on demand from a storage location. Often the term "paging" is used to mean the same thing. The term "paging" and associated terms like "page fault" are used throughout Eylon (Ex. 1005, Col 3-4). This is part of how virtual memory works on modern operating systems. Virtual Memory *was* a part of an ordinary computer science education in 2008, both at the undergraduate level as well as graduate level. I discuss undergraduate education in § A.. I discuss graduate education in § B.. It was, and remains a staple of computer science education, as can be seen by reviewing textbooks that were available in 2008 as well as current day. I discuss computer science textbooks in § C.. This is important because students are exposed to the concept of *memory mapped files*, which represents an alternative mechanism for accessing a file and does not require traditional read/write I/O operations.

- Page Cache/Buffer Cache — The term "cache" is used throughout Eylon, primarily to describe the persistent data cache maintained by the VFS (Ex. 1005, Col. 10, Line 49) and Christiansen, again to primarily to describe the persistent data cache maintained by the minifilter (Ex. 1006, 3). This is not the same as the page/buffer cache that is used by file systems on all modern operating systems, including Windows, Linux, and UNIX. Thus, there is an implicit assumption that the POSITA understands these varied uses of the term "cache" within the context of their particular operation. This is important because the complexities that arise in Dr. Houh's proposed minifilter relate to caching; if the Petitioner's POSITA does not understand the page cache/buffer cache (UNIX/Linux) or file system data cache (Windows) then that person will not not understand the challenges in implementing Dr. Houh's proposed minifilter. The understanding of the file system data cache and its integration with file system drivers is essential to implementing virtual file systems. This topis is one that is not covered in a standard undergraduate operating systems course because it does not make sense to do so given that file systems themselves are not considered to be a core part of such a curriculum. I discuss undergraduate education in § A.. Similarly, it is unlikely to be covered in a graduate level course because is specific knowledge that is not

even required to build a file system in UNIX or Linux. I discuss the page cache and buffer cache in UNIX and Linux in § E.. This understanding is important because application I/O is ordinarily satisfied by copying the needed data from the page cache, while paging I/O is ordinarily satisfied by retrieving the needed data from the actual storage, whether local or remote. The Board's decision states that these would be known by a POSITA to be equivalent, which requires the POSITA know how the page cache works, and how application I/O and paging I/O are satisfied to reach such a conclusion.

Thus, I maintain that the POSITA defined by the Petitioner is not one of ordinary skill, but rather one of extraordinary skill because:

- This person has experience with UNIX server systems as Eylon cannot perform any of its own functions without a server performing many of the tasks including the ordering of streamlets to the VFS. Eylon does not teach any such function on the client machine. Eylon further motivates his teachings because it permits the use of UNIX servers with Windows clients: "Furthermore, because the application server does not run application code but only delivers streamlets to the client, it can run on operating systems other than the application target operating system, such as UNIX derivatives or other future operating systems, as well as Windows-based servers." (Ex. 1005, p. 12, Col. 4, lines 62-67.) This is consistent with the educational background of the Petitioner POSITA definition because instruction was (and still is) done using UNIX-like operating systems. I discuss the background of someone familiar with file systems in the UNIX/Linux environment in greater detail in subsection G..

- This person has experience with Windows file systems. This is the field of the work of Christiansen. Christiansen uses the concepts and terminology of Windows file systems, which is substantially different than UNIX file systems. A person knowledgeable about UNIX file systems without Windows file systems experience would not recognize the language and teachings of Christiansen. The teachings of Christiansen are how to construct a file system mini-filter driver to permit caching of remote data contents; it relies upon a file system filter driver ("filter manager") and this is unique to the Windows file systems environment. I

discuss this more in subsection H.. Thus, the only realistic way a POSITA, as defined by the Petitioner, would have obtained this level of information is through direct experience, a process that would have required substantial time.

- This person has working knowledge of networking protocols and terminology because this person is experienced with streaming data mechanisms. Since the teachings of Eylon and Christiansen are related to lossless streaming technologies, this person would be familiar with TCP/IP or RPC over UDP/IP, both of which provide reliable transport. I discuss this more in subsection F..

- This person has working knowledge of encryption protocols commonly used in end-to-end communications, as well as storing data-at-rest, such as the U.S. Advanced Encryption Standard (AES) because this person understood the ordering of operations required by the system of the '808 Patent with respect to file system mini-filter drivers. I discuss this more in subsection J..

- This person has working knowledge of lossless compression protocols commonly used in storing data-at-rest, such as the Lempel-Ziv-Welch protocol, which such a person would know by experience is natively supported by the NTFS file system. I discuss this more in subsection K..

Finding a person with these skills in 2008 would have been a substantial challenge. In the period leading up to 2008 I was actively teaching course on developing Windows file systems and file system filter drivers. To the best of my knowledge, I was the *only* person teaching such courses in this time frame. Part of that education was translating the concepts of Windows file systems to those of UNIX because most of the developers with prior file systems experience attending my course had been involved in UNIX file systems development. There are multiple reasons for this: (1) UNIX was a commonly used server platform, though Linux became increasingly common in the period between 1998 and 2008; and (2) The bulk of active file systems research was done on

UNIX or Linux systems as there are hundreds of academic papers about file systems development prior to 2008 and very few were related to Windows; and (3) UNIX and Linux operating systems were available in source code form, which simplified the task of doing kernel development — this is unlike Windows where source code for the operating system has never been publicly available, which in turn creates considerable challenges building file systems due to the complex interactions between the file system and other operating system components.

Having reviewed the skills necessary in a POSITA to perform the analysis that the Petitioner calls upon them to do, it is my opinion that Dr. Houh does not meet this definition. Nothing in his CV indicates that he has the level of understanding of virtual memory and file systems required to understand the issues and trade-offs inherent in the system that he proposed. Throughout my analysis, I will point out that Dr. Houh failed to consider reasonable alternatives, the complexity of his proposed implementation, and the leaps in reasoning he injects into his analysis mean that his analysis is superficial and not rising to the level of understanding of the Petitioner's POSITA, let alone as an expert, as Dr. Houh represents himself to be.

B. Claims Construction

1. Generally

In this declaration, unless otherwise noted, I have used the Board's definitions to explain the basis for my proposed constructions.

2. "Extended Data"

The Board's decision requested that we explain construction of the term "extended data" as used in Figure 4 of the '808 Patent. Based upon the context in which it is used, The most reasonable construction of this would be "data unrelated to the contents of the file stream associated with the file stream by a file system, file system filter driver, or file system mini-filter driver." I based

this proposed construction on the ordinary use of this terminology with respect to Windows file systems.

Microsoft documents multiple ways in which information can be associated with various internal data structures related to file systems, file system filter drivers, and file system mini-filters in Windows. Usually these are described as "contexts" and are organized in such a way that the context can be retrieved relative to the object or data structure to which the context applies.

FLT_CONTEXT_REGISTRATION — this is a general purpose mechanism that allows file system mini-filters, using filter manager, to manage the extended data that can be associated with file system related information by a file system mini-filter. This includes: file context, instance context, stream context, stream handle context, section context, transaction context, and volume context. Each of these represents one form of context. (Ex. 2088, pages 206-208, 244)

FLT_CREATEFILE_TARGET_ECP_CONTEXT — this is a mechanism that allows associating specific extended data with a given create I/O operation in a file system mini-filter driver. (Ex 2088, pages 209-215)

IO_SECURITY_CONTEXT — this associates security context for a given create operation. Such extended data is necessary to notify the file system, file system filter drivers, and file system mini-filter drivers of the security entity creating a new or opening an existing file. This represents a concrete example of extended data (Ex. 2088, pages 229-243)

FLT_RELATED_CONTEXTS — this is a data structure in which a mini-filter is provided with the contexts for a given I/O operation. Each of these elements represents one example of extended data. (Ex. 2088, pages 248-249)

FLT_RELATED_CONTEXTS_EX — this is a data structure in which a mini-filter is provided with the contexts for a given I/O operation. Each of these elements represents one example of extended data. (Ex. 2088, pages 250-251)

3. "Data Stream"

Based upon my reading of the '808 Patent it is best to ground the definition of a "data stream" in the logic of encoding, like the Java example. This definition is consistent with the manner in which games themselves work. A vast array of graphical elements is often based upon a single common core, such as a wireframe, and various other elements, including textures and colors, that are then combined by the program to render specific details. From the asset developer's perspective, it is reasonable to consider each of these elements to be an "asset" and the combination of these related assets to be a useful collection. The background of the '808 Patent certainly draws from the world of computer gaming: "As an illustrative example, an exemplary user mode application according to principles of the invention may launch an application such as a game." (Ex. 1001, Col. 8, Lines 29-31.) Thus, an encoding of data with known structure to the application, represents a collection of individual assets, and that collection is equivalent to a "data stream". Such a stream would be complete when all the individual elements ("data packs") of the data stream were all available on the local computer. Note that, given the teachings of the '808 Patent, to be complete, a "complete data stream" need not consist of data that is even within the same file: "If only part of the data is found at one location, control will pass from location to location to retrieve all pieces of the data." (Ex. 1001, Col. 9, lines 3-5) and "The data does not need to be together, it can be distributed in pieces and segments across many local and/or remote sources." (Ex. 1001, Col. 9, lines 12-14)

I note that the term "data stream" is also not defined within the '808 Patent specification. I know of multiple contents in which the term "data stream" is known:

NTFS — the NTFS file system includes a concept of an "alternate data stream", where a single file is broken up into multiple distinct named pieces. The "default data stream" for NTFS is the name of the file along with "::$DATA" appended to it. Another commonly

occurring data stream in NTFS is known as "Zone_Identifier", which is used to store information about the source of information, such as a file downloaded from the Internet. Windows tracks that information to warn users when performing certain types of access to a file from a potentially untrusted source. The NTFS file system has supported "alternate data streams" since it was first released by Microsoft in 1993.

Networking — the networking community uses the term "data stream" to mean a sequence of bytes of data. Such data elements consist of arbitrary information transmitted over the network, typically using protocols such as TCP/IP.

Java — the Java programming language has an explicit use of the term "data stream" through the `DataInputStream` and `DataOutputStream` implementation of `DataStreams`. In such a system, specific data is encoded in a way that captures the structure of the underlying data so that the data can be stored or transmitted in such a way that it can be decoded later, without losing the structure of the data.

Data Mining — the Internet has created a vast array of continuous sources of information. "The storage, querying and mining of such data sets are highly computationally challenging tasks. Mining data streams is concerned with extracting knowledge structures represented in models and patterns in non stopping streams of information." (Ex. 2097)

Note that each of these uses of the term "data stream" originates from areas related to the subject matter of the '808 Patent, yet none agree with one another. However, I will note that *none* of these correspond to a construction in which a "data stream" is equivalent to a file: for NTFS, a file is a collection of data streams, each of which is ordinarily complete; for Networking, a data stream is a communications concept, unrelated to a given file, but typically representing some information to be interpreted by an application; for Java, it is a mechanism by which the structure of information is used to encode an arbitrary sized set of binary data elements (the "data stream") and such a set is "complete" if all the necessary elements are present; and for Data Mining, the

information is more like a seemingly endless stream of information that is unrelated to underlying storage.

4. "Complete Data Stream"

The term "complete data stream" is used in both Claims 1 and 14. This term is not a term of art and thus I turn to the specification to propose a construction.

In the Board's decision (Ex. Paper 8, p. 17) the Board concludes that the term "a complete data stream" corresponds to a file. However, I maintain that this construction is flawed because it excludes features taught by the '808 Patent.

The '808 Patent explicitly describes that a data stream may consist of elements from different files on the same computer system: "Upon receiving a request for a data pack (aka 'asset'), the user mode application looks up one or more media locations from a list, as in step 322. Such list of locations (i.e., addresses) may be part of the reference table or a separate list, file or other reference data source. By way of example and not limitation, the asset may be available from an accessible web server 324, ftp server 326, a CD or DVD 328, nonvolatile memory (e.g., flash media) 33 0, hard drive 33 2, and any other source 334. If no such source is identified or accessible, then the user mode application looks for another updated list, as in 1 o step 336, and returns control to step 322. If the asset is available from an accessible source, then a request to retrieve the asset is fulfilled, as in step 338, and receipt of the entire asset is confirmed, as in step 340. If any portion of the asset is unavailable from the targeted source, then control is passed to 15 step 324 to identify another source. When the entire asset has been received, the user mode *copies the data pack into memory* or saves it to disk and replies back to the kernel mode, as in step 342." (Emphasis added, Ex. 1001Col. 7, Lines 1-19.)

Note that the data pack in this case may come from a source other than the underlying file itself. This is logical: if the data pack is available from an existing local location, there is no benefit to copying it into a second location. The Board's proposed construction of "complete data stream" here would exclude this teaching. The '808 Patent calls attention to this advantage again

later: "This allows the package to be stored in whole, or in pieces, and on different types of media and locations." (Ex. 1001, Col. 11, Lines 43-44) Also "The delivered package itself may consist of smaller components of varying size and use depending on how it was prepared and how it may be requested. It may be a solid single source or a list of separate objects in a predetermined order at the same base location (e.g., 00001.zip, 00002.zip, 00003 .zip ...)" (Ex. 1001, Col. 9, Lines 54-59)

In addition, the NTFS file system supports multiple data streams per file. The board's proposed construction would require that all data streams of a single NTFS file be present for it to be a "complete data stream". This is inconsistent with the ordinary use of the term "data stream" with respect to files on a file system such as NTFS that permits multiple data streams.

Thus, the Board's decision to equate "complete data stream" with Eylon's term "file" is erroneous because it is inconsistent with the teachings of the '808 Patent as well as the ordinary use of the term data stream with respect to Windows file systems.

The '808 Patent used the distinguished term "complete data stream" is because it teaches that complete data streams need not be defined by the boundaries of a specific file.

The correct construction of the term "complete data stream" is "a distinguished collection of data packs".

5. "File Stream"

This is a term ordinarily used to specify an open instance of a file or, for a file system that permits multiple data streams within a given file or directory, an open instance of the stream. (Ex. 2088)

6. "Attach Extended Data to a File Stream"

Given the proposed construction of the term "file stream" (subsubsection 5.) the most reasonable meaning of the term "attach extended data to a file stream" for a file system mini-filter

would mean to use the filter manager function `FltSetStreamContext` (Ex. 2088, pages 831-833) to associate extended data ("context") with the specified file object.

7. "Data Packs"

The Petitioner has argued that application I/O and paging I/O are the same thing. This is in direct disagreement with their own documentation about these with respect to Windows and their position is disingenuous. It attempts to avoid one of the critical distinctions of the '808 Patent that differ from the cited prior art.

In summary: an application program can only issue application I/O. Only the operating system can issue paging I/O operations; these may be triggered as a result of application execution (a "page fault"), but it is ultimately the operating system that decides if and how to issue a paging I/O. For example, an application might be reading a single byte of data, but the operating system might convert this into a much larger read operation.

Application I/O represents specific information that the application has about the structure of data, which the '808 Patents teaches using the term "asset": "Counted assets are assets that have been tagged in a particular order for transfer, and may also refer to other counted assets to provide a dynamic loading tree that may change as the user operates the application. Common assets are objects that have been shared between multiple counted segments. In many cases, application operation will reuse the same sound, graphic, code or other resource in multiple instances. This allows for the media to be streamed with a counted object, but not be required to be streamed again from any medium." (Ex. 1001, Col 10., lines 27-36)

Thus, an asset consists of some number of bytes of information. That information may be present in multiple locations across multiple files. Because it is bound to the needs of the application, not the needs of the storage system, there is no reason to assume it has any well-defined relationship to blocks. The '808 Patent clearly teaches the idea that data may already be stored locally: "As an example, if 400 MB of a 5 GB total game is stored on a game CD, and 3 GB is stored on a peer to peer network, and 1.6 GB is stored on a webserver, a system according to

principles of the invention may summon each source to retrieve the data on demand. The data does not need to be together; it can be distributed in pieces and segments across many local and/or remote sources. By way of example and not limitation, part of a game (e.g., data for common sound effects, textures, game code, and the first 40 seconds of each video clip code) may be on a CD, while other data (e.g., data for landscape and dungeons) could be on a web server, with the rest of the foreign world data on an unpredictable peer to peer network. Thus, the invention allows seamless 20 play with all or only a fraction of the data actually present in any one location. By distributing the data, cost benefits are shared across the media and sources." (Ex. 1001Col 9, Lines 8-23)

In other words, one could go to GameStop, buy a version on optical media, bring it home and play it, with updated content being identified and downloaded from the computer of someone sharing your local network (peer-to-peer) as well as the content distribution web server of the game company. This is true, even if the location of specific assets moves around in updated copies of the files.

This is the beneficial teaching of the '808 Patent and the inventor chose to use a distinctive term and make clear that said distinctive term was tied to an ordinary concept drawn from the computer game field.

Thus, the construction must capture this insight that the structure of the data packs is related to the usage pattern of the intended application.

Paging I/O represents one or more pages in the virtual memory system, such as the file system data cache. I explain the basis for my claim that application I/O and paging I/O are distinct and separate in greater detail in subsection I..

The Petitioner argues that the '808 Patent describes a system in which *only* paging I/O is supported is a preferred embodiment. This is a misunderstanding of the '808 Patent because it is not a preferred embodiment. It is a **required** part of a viable system that would work as if the application were installed locally. For example, if I open up a game file with the Windows Notepad utility, it will be memory mapped by Notepad — I have used this example of "here is why you must support paging I/O in your file system driver" for more than two decades. Memory mapping

bypasses the normal read/write interface and instead relies upon the demand paging system of the operating system to load contents of the file, as needed. Eylon describes this process as "page faults" and that use is consistent with ordinary use in this domain.

What is not required is that a file filter driver support application I/O. According to Microsoft's documentation, there are four kinds of I/O that can be received by a file system mini-filter (Ex. 2088 pp. 227-228): paging I/O, cached I/O, DASD I/O, and non-cached I/O. Direct Access Storage Device (DASD) I/O relates to I/O operations directed at the underlying disk drive and are not of interest in this discussion. The remaining three define **two** general types of I/O. Application I/O which may be cached or non-cached, and Paging I/O, which is always cached. Thus, if an I/O request indicates it is a Paging I/O, the file system or filter driver knows that the I/O was initiated by the operating system. **Only the operating system may initiate paging I/O**. All Paging I/O is non-cached; this is true on Windows, UNIX, and Linux, as I explain in more detail in § E.. Further, at least the Windows APIs for sending paging I/O operations are not available to application programs as I explain in more detail in § I..

The normal practice for file systems on Windows is to allow the operating system to handle the file system data cache. (Ex. 2088 pp. 1392-1395, 1686-191). This simplifies their implementation and permits them to focus on handling non-cached I/O operations. File system mini-filter drivers are typically implemented to either handle application I/O or to handle paging I/O, but it is unusual for them to handle both.

The reason the '808 Patent in Figure 4 includes a description of how to handle paging I/O is not because it is *preferred* but rather because it is *required* for correct operation. A system in which only paging I/O is supported would lose the benefits of the asset aware model that is taught by the '808 Patent. However, a system in which paging I/O is *not* supported would lose numerous benefits: the ability to execute programs, the ability to utilize the computer's main memory (RAM) as a data cache, and the ability to support applications that use the memory mapped file APIs of all modern operating systems. Thus, handling paging I/O is a single element of a multi-element claim that relies on its other limitations for novelty.

The '808 Patent describes its embodiment in part: "If the entire data pack has not been received, as determined in step 418, a determination is made if the call is a paging I/O or a normal read. Paging (sometimes called swapping) is the transfer of pages between main memory and an auxiliary store, such as hard disk drive, which allows use of disk storage for data that does not fit into physical RAM. In a paging I/O, compressed packs are unpacked into allocated paging memory buffers at missing offsets in step 428, and control proceeds to step 432. In a normal read I/O, a new read request is created and sent for the same allocation to a lower filter driver and the original buffer is filled, as in step 438. Again, control is then passed to step 432." (Ex. 1001, Col 7, Lines 48-60)

The language here is *if* it is a paging I/O; there is no reasonable reading of this to exclude the or clause ("a normal read"). Thus, reading the specification to claim a paging I/O only system is in error and the rejection of my proposed construction is similarly incorrect.

Whereas in Eylon, uniform 4K framing is necessary to achieve optimal paging operation, the additional information, i.e., the "stremlet length" is redundant as it is always 4K. Paging can be achieved with that additional information but when achieved in accord with the '808 Patent as claimed this is a special case, wherein the general case allows data packs to be any selected length. For embodiments that allow sparsely packed files (e.g., regions of the file for which there is no corresponding data pack,) including a length is necessary in the table to know where the data pack ends, while in a system where the entire file is densely defined by its data packs, only the offset is required, as the length can be imputed by comparing with the subsequent offset.

If practiced as claimed in the '808 Patent, the description of a data pack consists of an *offset* and a *length* relative to a file. Thus, to characterize the reference to paging in the text of the '808 Patent as "paging I/O only embodiment" is inconsistent with the express language of the patent.

Support of paging I/O is a requirement of the file system on a modern operating system. A failure to support paging I/O yields a non-functional system because it does not support file execution or memory mapping. The former means it cannot be used to run programs and the latter means that it cannot be used with utilities such as Windows *Notepad*, the simple text editor. The system of Eylon preferably only supports Paging I/O. That is logical, because that is the least

effort to achieve Eylon's objectives. The system promoted by the '808 patent exploits the non-page-sized structure ("assets") of the files being used by the streaming applications to achive its objectives and thus uses application I/O operations. However, the system of the '808 Patent must also be functional - to allow Notepad and to permit execution of programs, so it must also support Paging I/O. The specification includes text to this point because satisfying a page fault in the '808 system requires computing a set of data packs that must be fetched, potentially from multiple different location, and might overlap with data packs that have already been fetched. That set must be sufficient to cover the entire data region described by the page. But this is done as a requirement to work properly within the operating system. A system that only supports Paging I/O sacrifices the very advantages that the '808 Patent teaches.

A system that is capable of supporting variable sized units, one specific embodiment in which, for whatever reason, the size of each data pack is identical is not excluded by my proposed construction. Just because the size *can* vary does not imply that it *does* vary.

In addition, the proposed construction excludes an embodiment of the '808 Patent in which the data packs are *sparse*. In other words, if a region of the given file does not have data that is used by the application, the embodiment in which we only describe the regions that do have data used by the application within them is excluded by the Board's proposed construction.

The idea that data packs might not be tightly packed is actually reasonable: it is common when building data structures for those data structures to contain unused regions that pad out to some boundary. Even in the paging I/O only system proposed by the Petitioner, there will often be areas of those pages that are never accessed. For example, modern compilers for the Intel x86 based processors will not split instructions across a page boundary because that can trigger memory access bugs when the underlying physical pages backing the virtual pages are not contiguous. In such a case, a more efficient implementation would split up the pages into variable sizes, excluding the unused regions within the page.

Applications cannot directly issue paging I/O operations. Instead, applications use read and write to access specific objects ("assets") that the application needs to perform its ordinary function. In UNIX, Linux, and Windows, an application issues a read call, which is ordinarily satisfied

by the operating system, not the file system (see subsection I.). The system described in the '808 Patent uses *application I/O* to identify the structure of the data within the file. It then constructs a definition of those elements by defining each element ("asset" or "data pack"). Note that there is no requirement here that all the data of the file be present: if the application does not access it, there is no reason to assign a given unused file region to a "data pack". This embodiment is disallowed by the adopted construction, despite the language of '808 that explicitly states data is organized according to "how and when the portions are used".

A file system mini-filter chooses what type of I/O it supports. Those choices are described in greater detail in subsection I. To summarize: to satisfy the preferred embodiment of the '808 Patent, the '808 mini-filter must handle both cached and non-cached I/O. Figure 4 of the '808 specification provides a description of how to handle *both* application I/O and paging I/O. If the mini-filter of the '808 Patent were to handle only application I/O it would not work properly because any data that is loaded using the paging mechanism, e.g., via a *page fault* would not be presented to the mini-filter and instead would be processed by the underlying file system driver. Since the file system driver does not have the entire data contents of the file, the data that writes into the newly allocated physical memory page will not contain the correct data contents. This is a *requirement* for any modern operating system due to security considerations.

Thus, the Petitioner's POSITA would know that it is far simpler to only handle non-cached I/O and that is sufficient for a system in which the smallest granularity is a page. This is why Eylon's preferred embodiment always uses 4KB (Ex. 1005,Fig. 7,) — it is the most common page size. "Instead, the application is streamed to the client's system in streamlets or blocks which are stored in a persistent client-side cache..." (Ex. 1005, Col. 3, Lines 52-54). Eylon does suggest this might vary, but provides no teaching of what other viable alternatives might be and thus does not teach the specific model taught by the '808 Patent.

Every one of Eylon's figures and examples uses this same fixed-sized unit. The specification of Eylon provides no guidance suggesting that streamlets should be variable sized units, nor that the size of those unit should be driven by the usage of that data by the application. A POSITA would understand that applications are demand loaded on a page granularity. A POSITA would

understand that the page size of the client computer may not be the same as the page size of the server computer. A POSITA would understand that it is optimal for the streamlet size to be the same as the client page size but that it would be correct to use a different streamlet size versus client page size even if it is less efficient than using the optimal page-size value.

The Petitioner's POSITA would have no reason to implement the more complex logic of variable size streamlets based upon the teachings of Eylon or any combination with the other prior art references.

Nothing in the teachings of the cited prior art teach the benefits of extracting application level data usage patterns ("structure"). This is at the heart of the problem that the '808 Patent is designed to solve: "Another key shortcoming of such systems and methods is an inability to prioritize delivery of components to expedite execution." In the context of the '808 Patent's specification, it is the *assets* that correspond to these components. The system taught by Eylon, with it's streamlet ("blocks"), has no sense of the internal structure. This makes sense, because it is the ordinary way in which file systems operate: the *contents* of the file are viewed as being opaque. This adherence to the ordinary principles of opaque storage made up of fixed sized blocks, Eylon uses the term "block" repeatedly in Figures 6A and 6B and only uses the term "streamlet" once (Ex. 1005, Figure Sheet 6). That "streamlet" (Step 620) is converted into a a *block* (Step 630) after the data has been validated. The block is discarded (Step 626) if the data fails its validation. This is consistent with how storage systems work: the content is opaque and simply divided into units that are optimal for some aspect of the computer system, such as the memory page size or an optimal storage unit size. Eylon further states: "In a preferred embodiment, each streamlet blocks corresponds to a data block which would be processed by the native operating system running on the client system were the entire application locally present. For example, standard Windows systems utilize a 4 k code page when loading data blocks from disk or in response to paging requests. Preferably, each streamlet is stored in a pre-compressed format on the server and decompressed upon receipt by the client." (Ex. 1005,Col. 3, Lines 59-65.) The language here is clear and unambiguous: the "streamlet" size should correspond to what is ideal on the client computer. There is no awareness of the structure of the application's data within files, no mapping

into assets, and no combination of local and remote storage elements to extract different parts of the file to the local computer. When the entire application is run from the local computer, there is no benefit for considering the very issues that the '808 Patent seeks to address.

This is markedly different than the teachings of the '808 Patent, where it clearly distinguishes itself from a block oriented system. The '808 Patent describes the ordinary function of the existing streaming on-demand system of Vinson, itself remarkably like Eylon: "Files of a program are broken into compressed, encrypted chunks in a preprocessing phase. This accomplished by opening the file, reading a block of chunk-size bytes, encrypting the block, compressing the block, calculating a unique file name for the block from the unique ID of the file and the block number, writing the just processed block to a new file in the specified directory of the runtime server using the file name previously generated, and then repeating for the next block in the input file until the end of file is reached. Thus, chunks are formed sequentially, one after another, comprising contiguous portions of the executable program."

The '808 Patent cites to the system of Vinson (Ex. 2067): "[a]n index contains the information used by a client-side file system driver (FSD) to compute which chunk to download at runtime as it is needed. The FSD also creates a virtual drive, complete with drive letter, and places each target program that is to be accessed under a top-level directory on that virtual drive. Access to a chunk of a remote target program that has not been downloaded to the client machine may be granted or not depending on the type of access being requested (i.e. which volume or file function is being executed), an ID of a process attempting access, and optionally the path specified in the request for those requests containing paths." (Ex. 1001, Col 1, Lines 56-67).

Vinson's system is thus quite like Eylon's system, including its use of a virtual drive, the use of fixed sized units to download data, etc.

Neither Eylon nor Vinson teach "data packs". They both use terminology ("streamlets" and "chunks") that their own specifications equate to "blocks". The '808 Patent uses the term "data packs" and, much like Eylon and Vinson tie their definitions to blocks, the '808 Patent ties its definition to "assets". There are multiple descriptions of asset instances in the '808 specification and *none* of them correspond to a fixed size unit.

To implement the system taught by the '808 Patent a file system mini-filter must handle both application and paging I/O for correct operation. A system that ignored application I/O and only processed paging I/O would have no need for the mapping table taught by the '808 Patent because there is a one-to-one correspondence between the offset in the file and the page on the remote system.

The purpose of the mapping table taught by the '808 Patent is to (1) improve the efficiency of the transfer by actually transferring exactly the data needed by the application; (2) permit loading on an asset basis, not on a page basis; and (3) provide a mechanism of eliminating download of duplicate assets. These benefits come with a price when implemented as a file system mini-filter: a file system mini-filter that does partial-page updates must process application I/O requests directly — they cannot be handled by the default behavior of the operating system implemented by the native file system.

Eylon teaches us the demand paging model: "During normal operation of an application, the Windows operating system will periodically generate page faults and associated paging requests to load portions of the application into memory." (Ex. 1005, Col. 4, Lines 10-13). That is all which Windows, or any other modern computer operating system, requires to support correctly working embodiments of Eylon.

Eylon's focus on page-based systems is, in fact, his preferred embodiment: "Preferably, each streamlet (when uncompressed by the client if necessary) has a size which corresponds to the standard 1/0 code page used by the native operating system." (Ex. 1005,Col. 12, Lines 1-3) This preferred embodiment is a normal storage preference for keeping the structure of file contents, which are meaningful to the application, opaque to the storage system.

Eylon also teaches us: "A set of streaming control modules are installed on the client system and include an intelligent caching system which is configured as a sparsely populated virtual file system ('VFS') which is accessed via a dedicated streaming 5 file system ('FSD') driver." (Ex. 1005, Col. 4, Lines 1-5). It is clear that Eylon is considering the Windows operating system platform, as noted above, because Eylon explicitly considers how Windows functions: "During normal operation of an application, the Windows operating system will periodically generate page

faults and associated paging requests to load portions of the application into memory." (Ex. 1005, Col 4. Lines 10-13). Eylon would have known that it is possible to utilize a file system filter driver to implement this functionality because in 1997 there was one book teaching both Windows file system and file system filter development and thus Eylon would have been taught both at the same time (Ex. 2017). Eylon teaches that the "virtual file system" and "streaming file system driver" work together to implement the client side functionality of the streaming application server. (Ex. 1005, Fig. 4). The combination "appear to the operating system as a local virtual file system which can be accessed by the operating system in the same manner as other data storage devices 190." (Ex. 1005, Col. 8, Lines 23-25)

It is important to note that a file system filter driver or file system mini-filter driver does not "appear as a local virtual file system which can be accessed by the operating system in the same manner as other data storage devices" because the very purpose of the filter model is: "In general, filter drivers ('filters') are kernel-mode drivers that enhance the underlying file system by performing various file-related computing tasks that users desire, including tasks such as passing file system 1/0 (requests and data) through anti-virus software, file system quota providers, file replicators and encryption/compression products." (Ex. 1007, Col 1, Lines 17-23) Also: "If the object is cached and the request is not to create a new object, modify an existing object, or open a directory, the request is directed to a local file system. Otherwise, the request is directed to a remote file system." (Ex. 1006,) Specifically, one constructs a file system filter driver to reside on top of an existing file system. Thus, the function of a virtual file system, as taught by Eylon is fundamentally different than the function of a file system filter driver or file system minifilter driver as taught by Pudipeddi or Christiansen.

Eylon does not explain why he chose a virtual file system embodiment. However he does suggest features that would not have been available to him if he had chosen to implement his system using a file system filter driver: (1) He relied upon the ability to utilize sparse storage — "[I]nclude an intelligent caching system which is configured as a sparsely populated virtual file system". (Ex. 1005, Col 4, Lines 2-3) Only the NTFS file system supported sparse files. Thus, had Eylon chosen a file system filter implementation of his solution, he would have limited himself to

systems that were using the NTFS file system. In 1998, the NTFS file system was still relatively new and many users continued to utilize the FAT file system. (Ex. 2052) This was particularly true in corporate IT environments that prefer "tried and true" technologies, especially something like the "new technology file system". (2) He had a specific desire to support custom formats and data encryption: 'To prevent unauthorized access to the streamlets (and thus various portions of the application files), one or more of the streamlets in the library, the streamlet map, and the application file structure can be stored in an encrypted or other 55 proprietary format." (Ex. 1005,Col. 11, Lines 52-56) Neither NTFS nor FAT supported data encryption. The Encrypting File System support (part of NTFS) was not added until Windows 2000 (Ex. 2104).

A VFS ensured that he could control access to the data, which is the same reason that Vinson used a similar virtual file system mode: "The present invention relates to network file Systems, and in particular, methods of allowing remotely located programs and/or data to be accessed on a local computer without exposing the remote program and/or data (henceforth collectively referred to as the target program) to indiscriminant copying. In addition, methods of limiting the length of time that the target program can be executed or accessed on the local computer are applicable. A persistent caching Scheme is also described which allows Subsequent accesses to the program or data to proceed with reduced download requirements even across different connection Sessions." (Ex. 2067, Col 1, Lines 13-25).0

Even in 2008, Eylon would have been motivated to implement his system as he originally taught because Windows XP continued to support using the FAT file system as the file system of the system partition, which did not provide encryption or sparseness.

An expert such as Eylon could have overcome these limitations by utilizing a file system filter driver, but it would have been substantially more complex and yielded a no more functional product than his original teachings provided. Thus, there is no motivation to implement Eylon as a file system filter driver.

I would note that a POSITA, reading Christiansen, would consider the most logical combination of Eylon with Christiansen according to the teachings of Christiansen: "With contemporary operating systems, such as Microsoft Corporation's Windows® XP operating system with an un-

derlying file system such as the Windows® NTFS (Windows® NT File System), FAT, CDFS, SMB redirector filesystem, or WebDav file systems, one or more file system filter drivers may be inserted between the I/O manager that receives user I/O requests and the file system driver." (Ex. 10060) In other words, the logical combination would be to place the mini-filter of Christiansen on top of either the virtual file system of Eylon. The Petitioner ignores this combination because it does not lead to the system that the Petitioner requires to argue that the '808 patent is obvious.

Instead, the Petitioner proposes a novel combination of Eylon and Christiansen, the first step of which is to convert Eylon to a legacy file system filter. Once this transformation is complete, the Petitioner can then apply the teachings of Christiansen to claim that this novel combination makes the '808 patent obvious.

Thus, the Petitioner has implicitly assumed the step of converting the system of Eylon to a file system filter driver. The Petitioner never explicitly states this step, yet once understood it is obvious based upon the Petitioner's own arguments: "Eylon, Christiansen and the 808 Patent are analogous art because they are in the same field of endeavor, i.e., managing I/O requests via filter drivers" (Petition, 2. "Reason to Combine", page 25).

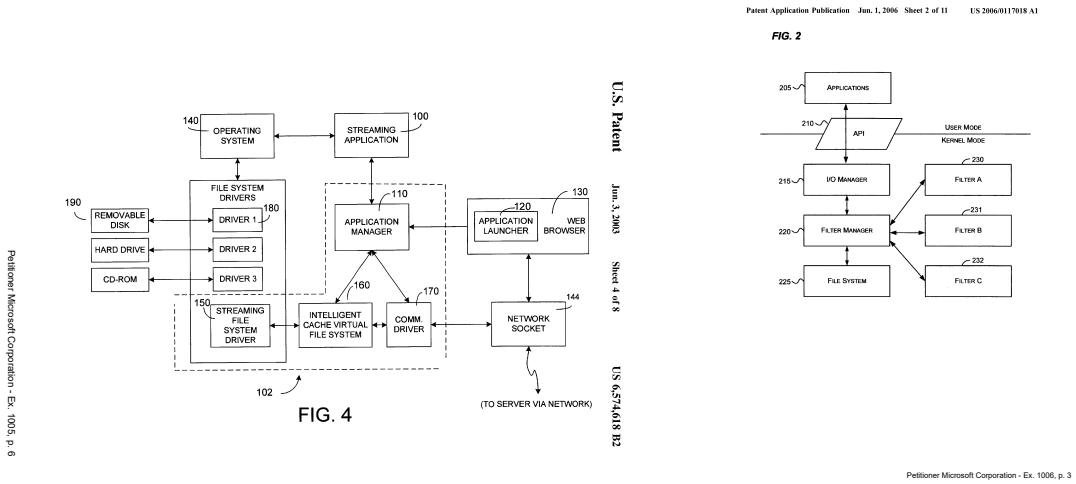

Figure 1: Comparison of Eylon and Christiansen: Eylon does not teach filters.

The term "filter" is never used in Eylon. Eylon Figure 4 (see Figure 1), which identifies the virtual file system components does not show anything labeled as a filter. Eylon Figure 7 (see Figure 2), which is a detailed view of the file system driver and VFS manager, similarly is devoid

of anything I could identify as a filter. Thus, the equivalence the Petitioner has drawn between the virtual file system of Eylon and the filters of Christiansen as being "in the same field of endeavor" is wrong.

Christiansen and Pudippedi teach us that filter drivers sit on top of file systems (previously quoted, Ex. 1007, Col 1, Lines 17-23 and Ex. 1006,) as shown in Figures 1 and 2. The Petitioner has created a false equivalence, one on which their subsequent analysis rests.

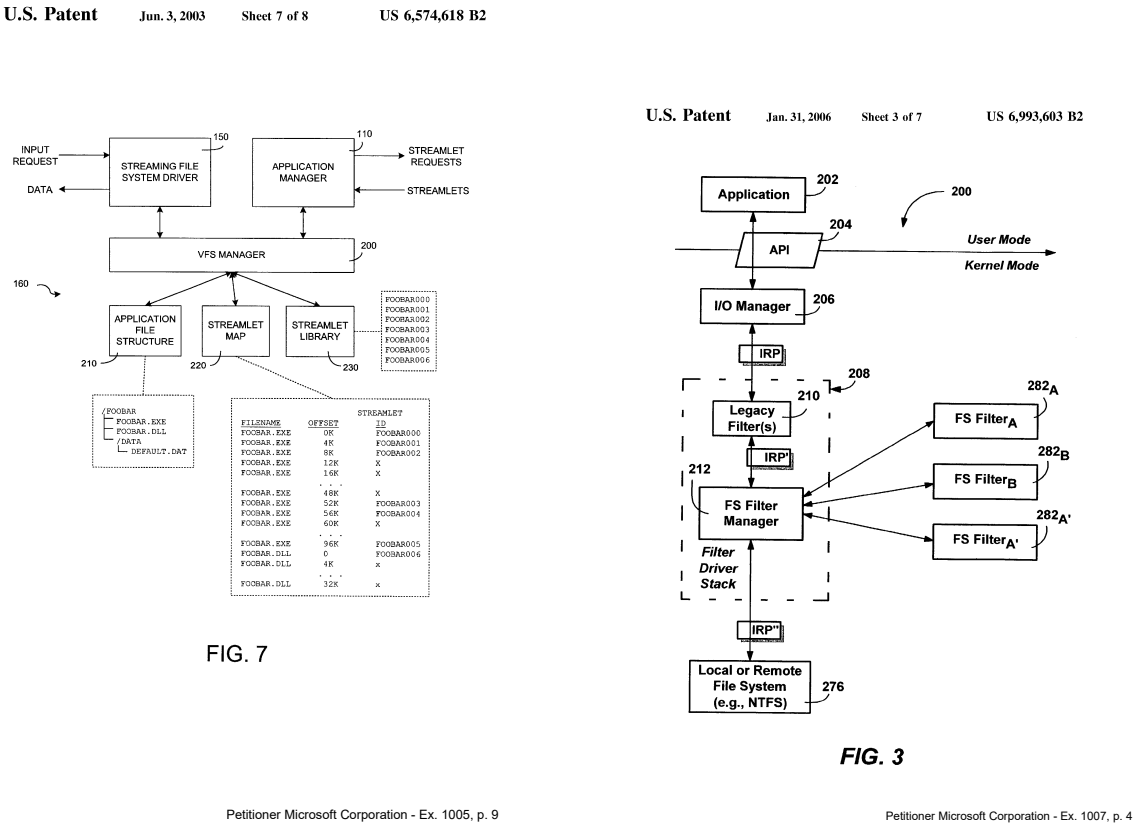

Figure 2: Comparison of Eylon and Pudipeddi: Eylon does not teach filters.

A file system can exist without *any* file system filter drivers — this is the configuration taught by Eylon. Filter drivers, cannot exist without file systems — this is the configuration taught by Pudipeddi and Christiansen. They are *not* equivalent. Christiansen and Pudipeddi teach us that filters sit above file system drivers.

This gives rise to the Petitioner's peculiar implementation of Eylon as a legacy file system filter driver with the teachings of Christiansen to implement the system taught by the '808 Patent.

Such a combination is not obvious, both because of the limitations that any filter driver based implementation imposes on such an embodiment, as well as the pragmatic reality that the challenging problems in implementing the system of the '808 Patent relate to its need to control the file system data cache because it wishes to handle the application level I/O, not just the OS paging I/O. "Upon receiving a request for a data pack (aka 'asset') ..." (Ex. 1001, Col. 7, Line 1). The concept of an "asset" is entirely an application level concept. There is no equivalent concept within the storage layer because it goes against the fundamental principles of modern storage systems, in which the structure of stored data is not known ("opaque").

My proposed construction for the term "data pack" remains: "portions of a data file that correspond to objects with specific information content meaningful to the streaming application." This does not preclude using fixed size pages **if** such a layout corresponds to the optimal use by the streaming application, but it allows for the way the term "data pack" is defined and used throughout the '808 specification.

In addition, this combined Eylon/Christiansen system does not realize the benefits of using a more flexible division of the file based upon its usage by the application because the application usage information is lost when such a system only processes non-cache I/O operations. This paging I/O only file system mini-filter does not have an altitude restriction and will work properly even with a layered encryption and/or compression filter on top of it.

The analysis of this term in the Board's decision (Paper 8, pp 21-22) ignores the '808 Patents use of an additional mapping layer known as the memory location address table (**MLAT**). This term is used four times in the diagrams (Ex. 1001, Figure 4)) and seven times in the specification (Ex. 1001: Col. 7, Line 29; Col 7, Line 33; Col 7, Line 36; Col. 7, Line 63; Col. 8, Line 44; and Col 8. Line 59), suggesting its use is not incidental. The function of the MLAT is to define the captured structure of the data packs themselves and then to show where those data packs are located. To do this, the specification clearly describes the information present in the MLAT:

name and path — this is the name and path of the data pack (Ex. 1001, Col. 7, Lines 27-28); and

identification — this is an identifier for the given data pack (Ex. 1001, Col. 7, Lines 32-34); and

fulfilled – this is an indicator if the data pack contents are stored within the name and path of the respective file (Ex. 1001, Col. 7, Lines 34-37); and

serviced — this is an indicator if the data pack has been serviced (Ex. 1001, Col. 8, Line 44 to Col. 9 Line 23); and

source — this is a list of one or more sources for the given data pack (Ex. 1001, Col. 8, Lines 60-62).

Thus, the MLAT provides a description of where the data packs or assets of the given file are presently stored: in the underlying file, in a different local storage location, or in a remote storage location. The MLAT is used to describe the software package, its assets, and their storage location, as this is important to identifying assets that are duplicated and stored across the various locations, as taught by the '808 Patent. To that extent it is not tied to any specific file within the software, it is maintained as a separate persistent data structure and it is not tied to filter manager contexts, though it may be referenced by zero or more filter manager contexts.

To satisfy an I/O request, the data region being read is satisfied by using the information from the MLAT, with the specification indicating three distinct ways in which this can be accomplished: reading it from the local file, reading it from a remote storage location, or reading it from an alternative local location. These mechanisms may be used individually or in combination as appropriate for the given I/O operation.

The *expectation* is that an application I/O will typically correspond to a single data pack, as the use of the application's structure for its own information is a key insight of the '808 Patent's teachings. For paging I/O, the data is being loaded by the operating system, not the application, and thus this still must ensure correct performance. In such a case, the '808 system will construct

a list of the data packs required. For data packs stored within the underlying file, that portion of the paging I/O can be satisfied from the local storage.

Similarly, in the unusual instance of a write operation (Ex. 1001Col 7, Line 66 to Col 8 Line 12) the MLAT is marked to indicate the data has been written locally ("fulfilled"). The specification teaches that such operations may require multiple steps to use the data that has already been recorded, with each MLAT entry being updated to indicate that the data is locally present within the underlying file.

Thus, the definition of "data pack" as identified by the Board fails to properly capture the use of this term in the '808 specification and claims because:

- It loses the concept of the data pack corresponding to an element of data that has specific meaning attributed to it by the application. This is inconsistent with the '808 specification, which teaches that "data packs" are defined by the application as identified by the application usage pattern.

- It suggests that the '808 Patent claims a paging-only I/O system as a preferred embodiment, which is contrary to the teachings of the '808 patent, because the '808 Patent teaches an embodiment that includes both application I/O and paging I/O. As I have previously noted, a system that teaches only paging I/O support, which is consistent with the fixed blocks preferred by Eylon, will work correctly but fail to achieve the optimizations of the system taught by the '808 Patent. Thus, implementing a system with only paging I/O — something that the '808 Patent *does not* describe as a "preferred embodiment" would work, but would sacrifice the very benefits of using application defined "data packs" that form the core teaching and primary benefit of the '808 Patent. Similarly, as I have previously noted (§ 7.), a system that teaches only application I/O support, which is not taught by the '808 Patent specification, would limit functionality by disallowing executing programs — which require paging I/O — and eliminate the use of the file system data cache, which degrades performance.

- This construction explicitly runs afoul of the embodiment described in the '808 Patent specification: "Assets (i.e., files or file portions) are broken up into three categories: Counted, Common and ETC. Counted assets are assets that have been tagged in a particular order for transfer, and may also refer to other counted assets to provide a dynamic loading tree that may change as the user operates the application. Common assets are objects that have been shared between multiple counted segments. In many cases, application operation will reuse the same sound, graphic, code or other resource in multiple instances. This allows for the media to be streamed with a counted object, but not be required to be streamed again from any medium. ETC objects are objects that were not accounted for, but are part of the software itself. These are things that may not be consumed in the normal operation, but exist with the package such as text files with release notes or for any other purpose that go with the pack age, but are not necessarily used by the package itself." (Ex. 1001, Col. 10, lines 26-41) No streamlet ("block") oriented mechanism can achieve the described embodiment because the structure of the information is lost. Nothing within the teachings of the cited prior art teach the use of the application-defined structure of the data as the basis of the size of each individual streamed element ("data pack"); a construction that loses this key distinction is fundamentally flawed because such a construction abandons a key feature and advantage of the method taught by the '808 Patent. As an additional example of this distinction, which is not present in the cited prior art, the '808 patent teaches us the distinction between data that is consumed by the application ("assets") versus data that is included but *not* consumed by the application ("release notes"). The cited prior art does not teach this distinction, which is consistent with the cited prior art's treatment of data as being opaque to the streaming system.

This definition properly captures the usage within the '808 patent because:

- It explicitly captures the "asset" concept as defined in the specification. The term "asset" is one that directly relates to specific information content, as defined by its use in the streaming application.

- It is consistent with the specification's distinction between application I/O and paging I/O, and why it supports both. For example, the '808 Patent states: "The minifilter receives each I/O request from the application, references a table that includes at least one address where each data pack required to fulfill each I/O request is located, and determines if the data pack has been streamed to the system." (Ex. 1001, Abstract)

- It is consistent with the specification's description of the three types of assets. This is consistent with the primary benefit of the teachings of the '808 Patent, its distinction of these types, and the benefits of identifying them. The proposed construction is consistent with the language of the '808 Patent specification, unlike the proposed construction adopted by the Board.

Thus, the construction I have proposed for the term "data pack" is consistent with the original equivalent "asset", captures the idea that the specifics of the "data pack" are tied to the application's use of the information corresponding to the specific "data pack" and has a distinct type, which is again tied to how the data is used by the application. Further, the Board's assertion that it disallows a "preferred embodiment" is incorrect because the board's argument that a paging I/O only system is a "preferred embodiment" is in error and is not supported by the specification.

The construction proposed by the Board eliminates the key teaching of the '808 patent: that there is structure to the file data, known to the application, and useful in constructing a streaming on-demand system.

Even though both are used in dividing up a file into pieces for streaming, the actor defining those divisions is different: the '808 Patent focuses on using the usage of the data by the application to define those pieces, while the cited prior art work focuses on allowing the storage system to define those pieces. They are certainly the same pieces, but they are as different as a traditional jigsaw puzzle might be to the division of the same graphic image divided into a grid of pieces.

It is my opinion that the construction I have proposed is consistent with the language of the '808 Patent and the construction the Board has proposed is inconsistent with the language of the '808 Patent. A POSITA, using the Petitioner's definition, would have understood the

distinctions between application defined data packs and storage defined streamlets and would not have concluded they were equivalent.

8. "Minifilter"

I propose that the term "minifilter" mean "a filter that is managed such that it is called only for operations for which it has registered."

I would note that the term "registered" in this context can be both an affirmative step, e.g., by providing a pre-callback or a post-callback function, as well as a negative step, e.g., by indicating that a particular class of operation is to skip the filter, such as `FLTFL_OPERATION_REGISTRATION_SKIP_CACHED`, which is defined by Microsoft as an option that ensures the filter manager does not invoke the minifilter for cached I/O operations. (Ex. 2088 pp. 227-228)

9. "Filter Minifilter Driver"

I would agree that the term "filter minifilter driver" should read to mean "encryption minifilter driver," based upon the context in which it is used in the '808 specification and claims.

10. Other Claim Language

In addition to the other claims constructed, I propose a construction for the term "reference a table that includes names and paths with at least one offset address and length where each data pack required to fulfill each I/O request is located, said minifilter being further configured to determine if the data pack has been streamed to the system."

Specifically, the reference table taught in the '808 specification is itself critical to the proper understanding of the '808 claims. In the specification, this is referred to as the "memory lookup audit table (MLAT)" (Ex. 1001, Col. 7, Lines 28-29) It's key properties are that it:

- Is dynamic: "Assets (i.e., files or file portions) are broken up into three categories: Counted, Common and ETC. Counted assets are assets that have been tagged in a particular order for transfer, and may also refer to other counted assets to provide a dynamic loading tree that may change as the user operates the application." (Ex. 1001, Col 10, Lines 26-31)

- Uses the path, file name, offset, and length of the I/O operation to determine the set of data packs that need to be present to satisfy the given I/O request. (Ex. 1001, Fig. 4, and Col. 7, Lines 30-40)

- Contains information indicating if the needed data packs are present in the underlying file system file. (Ex. 1001, Col. 10, Lines 36-38)

- Is updated when an application writes data to the underlying file (Ex. 1001, Col. 10, Lines 61-65)

- It permits a many-to-many mapping of asset packs to storage locations. (Ex. 1001, Col. 11, Line 59 to Col. 12, Line 14)

C. Alleged Obviousness over Christansen and Eylon, Claims 1-20

1. Overview of Christiansen The system proposed by Christiansen is one possible realization of "client side caching," in which data stored on a remote server can be similarly stored on a local computer. It seems primarily motivated by the needs of data centers and to that extent describes what one might see in a content distribution network based upon a network file system or redirector. It implements its client side functionality using a file system mini-filter driver that monitors the redirector and determines if and when it should cache data fetched from the remote file system into a local persistent storage location (the "cache").

2. Overview of Eylon The system of Eylon is a one in which application programs are packaged and distributed from a specialized application server to clients, such as Windows clients. It relies upon the server to predictively push content from the server to the client in an attempt to optimize the performance of the client. The client component is implemented as file system driver combined with a virtual file system. The file system driver described is like a network redirector or local media file system. The virtual file system driver is a mechanism for providing storage management that is used by the file system driver to store blocks of the streaming application send to the client from the server and to retrieve blocks of the streaming application from the local store to satisfy the access requirements of the streaming application.

3. Independent Claim 1

(a) Preamble [1.a] "A streaming on demand system comprising"

The Petitioner asserts that the system of Eylon teaches this limitation of the '808 Patent Claim 1 (and the similar limitation of Claim 14). The Petitioner's analysis is flawed because the ordinary use of the term "on demand" indicates a system in which the consumer of something triggers delivery. A comparable example might be how purchases from Amazon work: I can go to Amazon's website and purchase goods. This is clearly an example of an "on demand" system. On the other hand if I *subscribe* for delivery, Amazon delivers this to me on a fixed schedule. Amazon might choose to substitute some other comparable product for me, delay delivery, or even cancel my subscription without any action on my part. That is not an "on demand" system, it is a system driven by an external process.

Eylon teaches a system like my Amazon analogy: the client can order what it wants but Eylon emphasizes that utilizing the server to push content is a primary benefit of the system: "To improve responsiveness of the system when the application is not fully loaded in the cache, a predictive engine can be used on the server. Based upon information gathered from user interaction with the application at issue, a statistical usage model can be built which tracks the order in which users access the various streamlets. When a client starts a streaming application, an initial set of streamlets sufficient to enable the application to begin execution is forwarded to the client.

Additional application streamlets can then be actively pushed to the client in accordance with the predictive model and possibly additional data forwarded to the server from the client while the application executes. As a result, many of the needed streamlets will be present in the client-side cache before they are needed." (Ex. 1005,Col. 4, Lines 37-50) This is akin to the subscription model, in which it is the server that determines what content to send to the client computer. This is different than the claims of the '808 Patent, which rely entirely upon the client (the "computing device") to request content. The benefits of the predictive approach taught by Eylon are achieved in the '808 Patent by placing the predictive loading process on the client ("computing device"). This approach is superior to Eylon's because it allows customization of the behavior of the individual computer system. Thus, the '808 Patent achieves the advantages of predictive loading as described by Eylon but does so by using a pure client-driven system to do so.

Christiansen teaches a system in which the objects to be stored on the local computer may be determined by a remote component (the "cache server"): "The content server application 605 may comprise various components (not shown) that may independently access objects from the remote and local file systems 615 and 625. In some embodiments, no single component of the content server application 605 is aware of which objects all the components have requested or where these objects reside (e.g., locally or remotely). To monitor which remote objects are being requested, the filter 610 may monitor I/Os to and from the remote file system 615. Periodically, the filter 610 may send a list to the caching service 620 that includes the names of objects accessed from the remote file system 615. The caching service 620 may then use the list sent by the filter 610 to determine which objects from the remote file system to cache on the local file system 625. This determination may be made based on a policy set by an administrator or the like which may include frequency of access to the object, size of the object, or any other caching rules." (Ex. 1006, 9-50) This is not teaching a "streaming on demand system" it is teaching a server mediated system.

Neither Eylon, nor Christiansen teach a streaming on demand system: both systems teach server mediated systems in which cached content are driven or dictated by other components within the system. The combination of Eylon with Christiansen does not rectify this, either, with

the two teachings reinforcing their respective teachings of using servers to control the use of the cache and the prefetch logic.

Thus, the cited prior art references do not teach this limitation of the '808 patent.

(b) Limitation [1.b.i] "... an on-demand requester object installed on a computing device, ..."

In the Summary of the Eylon patent, Eylon admits that not all the elements are on the client:

"Advantageously, the present architecture described enables a positive user experience with streamed applications without requiring constant use of broadband data links. In addition, and unlike remote-access application systems, the streaming applications execute on the client machine and server and network resources are utilized for delivery only. As a result, server and network load are reduced, allowing for very high scalability of the server-based application delivery service which can deliver, update and distribute applications to a very large number of users, as well as perform centralized version updates, billing, and other management operations. Furthermore, because the application server does not run application code but only delivers streamlets to the client, it can run on operating systems other than the application target operating system, such as Unix derivatives or other future operating systems, as well as Windows-based servers. In addition, the application streaming system is source code independent. Applications are packaged automatically on the server in accordance with an analysis of the sequence in which the application is loaded into memory during execution. No changes to the application itself are required." (Ex. 1005, Col. 4 Line 51 through Col. 5 Line 5)

Note that the "computing device" of Claims 1 and 14 of the '808 Patent is exclusively responsible for driving the behavior of the system. In the specification there is no requirement that there be a remote server, describing local storage alternatives (e.g., local optical media, non-volatile memory such as flash memory) and non-server alternatives (peer-to-peer data sharing) as well as existing servers, such as a web server. This limitation of the '808 Patent is why only the client computer ("computing device") requires a specific component — the on demand requester object.

Thus, this limitation of the '808 Patent is distinguished from Eylon. In Eylon there is no on demand streaming system without an intelligent server because the server defines the streamlets, identifies the correct download order, and pushes the data to the client based upon its predictive model. In the '808 Patent there is no intelligent server, application behavior defines the data packs, and the client pulls data from local *and* remote sources based upon client-specific knowledge.

(c) Limitation [1.b.ii] "said on-demand requester object being configured to receive I/O requests on behalf of an application for which data is available in data packs for streaming delivery"

In subsection I. I explain the distinction between application and paging I/O. In summary, paging I/O is exclusively initiated by the operating system and *not at the direction of the application* and thus is different than an I/O request on behalf of an application because all operating systems include components for which the paging I/O operations are unrelated to the I/O operations of applications. Further, an application I/O may be entirely satisfied by the file system data cache or page cache (the specific terminology depending upon the operating system) as I describe in § E. In such a case, the paging I/O cannot be used to determine the application's I/O behavior.

Thus, I disagree with the Petitioner's assertion that this limitation is taught by Eylon. Eylon teaches intercepting paging I/O operations: "During normal operation of an application, the Windows operating system will periodically generate page faults and associated paging requests to load portions of the application into memory" (Ex. 1005, Col. 4, Lines 1-3). A POSITA would have understood that paging I/O operations are never initiated by application programs directly. Furthermore, paging I/O operations are routinely done in an arbitrary context that is not directly tied to a single application. Finally, paging I/O operations are done in fixed sized units, typically 4KB, because that is the size of a "page" for Intel x86 architecture CPUs.

There are specific reasons why implementing paging I/O in the file system of Eylon is logical: (1) it provides the simplest implementation because it does not require a VFS that handles application I/O — the operating system will be invoked to satisfy application I/O operations by retrieving the contents from the file system data cache, which will trigger a paging I/O when the necessary data is not present in the file system data cache; (2) pages generally have a good correspondence to

storage blocks (e.g., NTFS defaults to using 4KB storage allocation units and modern hard disks use 4KB sectors); and (3) the implementation of the bookkeeping necessary for a system using a single fixed size block is simpler than one that uses variable size blocks, e.g., a simple bit field versus a potentially complex sparse range list.

A POSITA, familiar with how operating systems and file systems work would be motivated to keep the implementation as simple as possible because doing so minimizes the development time and the opportunity for complex bugs to arise during development. A POSITA familiar only with streaming technology would be motivated to keep the implementation as simple as possible to minimize the "learning curve" of building a file system. In the time of the '808 Patent, a POSITA would have likely been motivated to implement the system of Eylon using a user mode file system. I discuss user mode file systems in subsection L..

The Petitioner argues that Eylon teaches the size of a streamlet could be something other than a page. The Petitoner's claim that Eylon teaches variable size streamlets is inconsistent with the teachings of Eylon because Eylon emphasizes that the preferred object size is driven by the memory system of the computer, not the size of data structures within the application: " If one or more of the requested streamlet blocks are absent from the VFS, a fetch request is issued to the server ..." (Ex. 1005, Col.4, Lines 25-27) – blocks are defined by the storage system, not the application. A POSITA would have understood that the reason for this variance between the client machine page size and the streamlet size is because the page size of the system on which the streaming application is being executed might differ from the page size of the system on which the streaming file is stored: there is no uniform page size across systems and it is most logical that Eylon wanted to ensure that this possibility was captured: "[E]ach streamlet blocks corresponds to a data block which would be processed by the native operating system running on the client system were the entire application locally present." (Ex. 1005, Col 3, Lines 60-63). If the entire application were present, as taught by Eylon, the existing demand paging system would be a good solution. However, the system taught and claimed by the '808 patent solves the problem where the program and its data are not entirely present and instead of behaving in the default fashion of demand paging, it specifies that the process is drive by application I/O. Note that paging I/O

is only initiated by the operating system and thus this limitation does not apply to paging I/O because it is not an I/O request on behalf of an application.

In addition, Eylon explicitly identifies the use of demand paging as one of its advantages: "Advantageously, the streaming mechanism leverages a conventional property of the operating system which uses memory mapped files to avoid having to load complete files into memory to thereby avoid having to stream more than a small portion of an application to a client before initiating execution and to transparently manage the subsequent operation of the application without having to modify the application code or data in any way." (Ex. 1005Col. 12, Lines 57-64). Thus, Eylon explicitly relies upon the behavior of the demand paging system to achieve at least one advantage.

Eylon teaches: "Preferably, each streamlet (when uncompressed by the client if necessary) has a size which corresponds to the standard 1/0 code page used by the native operating system. " (Ex. 1005,Col. 12, Lines 1-3).

Eylon consistently ties the size of streamlets back to a single fixed size, explaining its advantages in some places and merely stating a preference in others. A POSITA reading Eylon would not understand the benefits of using the more complex system taught by the '808 Patent.

Thus, Eylon does not teach this limitation.

Christiansen does not use the same terminology as Eylon or the '808 Patent. Instead, Christiansen adopts the term "object": "Briefly, the present invention provides a method and system for caching remote objects locally." (Ex. 1006, Col 1, Paragraph 4). My understanding of the meaning of this term, based upon its usage within the specification, an "object" appears to be a file, or perhaps a data stream of a file stored in a file system that supports multiple data streams: "In one aspect of the invention, a filter monitors requests and reports to a caching service names of objects accessed on the local and remote file systems." Neither the streamlets of Eylon, nor the "data packs" of the '808 Patent seem to have a file system type name, as described by Christiansen. Thus, it is my opinion that a POSITA reading Christiansen would not gain any appreciation of the application-driven "data pack" style division of files into data pack/assets, nor an appreciation of the storage-system driven "streamlets" style division of files into streamlets/blocks. Thus, the

combination of Eylon and Christiansen provides no motivation to use variable size streamlets, in which the size of the streamlet/block/data pack/asset is defined by the application's usage pattern. Nothing in the language of either prior art reference refers to variable sizes.

Elsewhere I have explained why a POSITA would not be motivated to combine Eylon and Christiansen. In this instance, I would note that the teachings of both Eylon and Christiansen that I have cited here require the presence of a server. The '808 Patent claims do not.

Finally, a POSITA wishing to combine Eylon with Christiansen would be motivated to keep the implementation simple to minimize development time, software defects, and the corresponding cost and business risk associated with them; thus a fixed size block scheme, limited to only handling paging I/O driven by the size of the anticipated storage and/or memory system, would be the least risky option.

Thus, there would be no motivation for a POSITA to implement the variable asset pack model taught and claimed by the '808 Patent based upon the teachings of the cited prior art.

(d) Limitation [1.b.iii] "said on-demand requester object being responsive to application I/O requests"

In subsection I. I explain the distinction between application and paging I/O. In summary, paging I/O is exclusively initiated by the operating system. An application I/O may be entirely satisfied by the file system data cache or page cache (the specific terminology depending upon the operating system) as I describe in § E.

Eylon teaches a system that is responsive to paging I/O operations, which are defined by the computer hardware and operating system as fixed size blocks. For example, Figure 6A and the corresponding text in the specification indicate this is the normal expected behavior of the system: "Turning to FIG. 6A, when a Page Fault exception occurs in the streaming application context, the operating system issues a paging request signaling that a specific page of code is needed for the streaming application." (Ex. 1005, Col 10, Lines 34-37) "When the FSD 150 receives a paging request (step 600), it passes the request to the FSD 150, and thereby to the VFS 160 which determines if the desired data is available (Step 602). If the page is present, it is retrieved

and returned to the operating system. (Step 608). In many cases, as a result of the predictive streaming, the desired page will already reside in the Cache/VFS 160." (Ex. 1005, Col 10, Lines 43-49) "When the desired page is not available, a fetch request for the needed data is sent through the communication driver 170 (or an alternate route, depending on implementation specifics) to the server 12. (Step 604)." (Ex. 1005, Col 10, Lines 53-56) "Preferably, each streamlet (when uncompressed by the client if necessary) has a size which corresponds to the standard I/O code page used by the native operating system." (Ex. 1005, Col. 12, Lines 1-3) "The streamlet map 220 is shown in a basic table format for simplicity. Also shown in the sample streamlet map 220 are absent code pages." (Ex. 1005, Col. 12, Lines 16-18)

The '808 Patent teaches a system that is responsive to application I/O and paging I/O; this is one of the advantages of the '808 teachings over the prior art because using awareness of how data is used by the application. Thus, this limitation is distinguished from the teachings of the prior art because the responsiveness of the system is related to the application I/O, rather than the paging I/O.

Christiansen is silent as to the issue of paging I/O versus application I/O and thus combining Eylon with Christiansen still does not teach this limitation of the '808 Patent.

Thus, it is my opinion that a POSITA would have known the difference between application I/O and paging I/O, and would have understood the additional insight of the '808 Patent to exploit the understanding of the internal structure of the file to improve the efficiency of the on-demand streaming delivery service taught by the '808 Patent. It is my opinion that a POSITA reading the cited prior art work would not have understood this insight and would not have appreciated Limitation 1.b.iii.

(e) Limitation [1.b.iv] "said application I/O requests corresponding to sequential and non-sequential data packs"

I disagree with the Board's decision that this limitation is taught in the prior art because:

The cited prior art teaches a system in which the *server* drives the pre-fetching behavior of the streaming on-demand system: "To improve responsiveness of the system when the application is not

fully loaded in the cache, a predictive engine can be used on the server. Based upon information gathered from user interaction with the application at issue, a statistical usage model can be built which tracks the order in which users access the various streamlets. When a client starts a streaming application, an initial set of streamlets sufficient to enable the application to begin execution is forwarded to the client. Additional application streamlets can then be actively pushed to the client in accordance with the predictive model and possibly additional data forwarded to the server from the client while the application executes. As a result, many of the needed streamlets will be present in the client-side cache before they are needed." (Ex. 1005, Col. 4, Lines 37-50) Thus, the system of Eylon uses a server-driven, rather than application driven pre-fetch behavior. Such a system must, by its nature, consist of a streaming application server that has the logic necessary to "push" contents to the client. The '808 Patent instead teaches that data comes from providers that are not specialized streaming application servers: "By way of example and not limitation, the asset may be available from an accessible web server 324, ftp server 326, a CD or DVD 328, nonvolatile memory (e.g., flash media) 33 0, hard drive 33 2, and any other source 334." (Ex. 1001, Col. 7, Lines 5-9) None of these data providers have an existing mechanism for pushing unsolicited content to the client.

In addition, the system taught by the prior art does not focus on the meaningful elements ("data packs") being used by the application because it combines them into fixed sized units ("streamlets"). Eylon repeatedly uses the term "streamlet" and "block" interchangeably, such as in Figure 6B, where Step 620 uses the term "streamlet" and 626 refers to it as "streamlet block", while 630 refers to it as "streamed block" The example in Figure 7, shows the streamlet map as a series of offsets and streamlet IDs; it does not include a length for the streamlet, which requires the streamlet be fixed length. Similarly, Figure 8, shows a file difivded into fixed size 4KB units, which is consistent with the use of the term "streamlet" throughout Eylon's specification. Eylon clearly realizes this is a normal function of executable programs, such as when they state: "During normal operation of an application, the Windows operating system will periodically generate page faults and associated paging requests to load portions of the application into memory." (Ex. 1005,

Col 4, Lines 10-13) Since page faults perform read operations in page size units — typically 4KB, as taught by Eylon — using a fixed size streamlet would be obvious to a POSITA.

This consistent teaching that streamlets of fixed size units of data permeates the text of Eylon:

- "More specifically, the application to be executed is stored as a set of blocks or 'streamlets' (parts into which the application has been divided) on a server. In a preferred embodiment, each streamlet blocks corresponds to a data block which would be processed by the native operating system running on the client system were the entire application locally present." (Ex. 1005, Col. 3, Lines 57-63)

- "When the application manager 110 is started, it is provided with information that identifies the streaming application to execute and also indicates which streamlets or blocks of the application are required for initial execution." (Ex. 1005, Col. 9, Lines 39-43)

- "When the application manager 110 is started, it is provided with information that identifies the streaming application to execute and also indicates which streamlets or blocks of the application are required for initial execution." (Ex. 1005, Col. 12, Lines 1-3)

I disagree that this limitation is taught in the prior art. Eylon teaches a system in which predictive fetch is driven by the server: "Additional application streamlets can then be actively pushed to the client in accordance with the predictive model and possibly additional data forwarded to the server from the client while the application executes. As a result, many of the needed streamlets will be present in the client-side cache before they are needed." (Ex. 1005, Col 4, Lines 45-49). This is not the on demand system driven by the application running on the computing device taught by the '808 Patent: "The client will queue any related common objects that are used with a counted object or requested object. It will then queue the next related counted object and repeat this process until all objects have been sent. Once they are completed, it will send over any ETC objects not accounted for." (Ex. 1001, Col 11, Lines 61-65).

Eylon teaches a "one size fits all" model to push content from the server to the client. The '808 Patent teaches a client-centric model to pull content from the relevant storage location (local or

remote) based upon the needs of the individual client. These two systems are markedly different because Eylon was focused on business applications, which mostly involved streaming cod, while the '808 Patent was focused on games, which mostly involved streaming data. Using Eylon in the context of gaming would be subject to the performance bottlenecks described by the '808 Patent which would degrade game performance.

Thus, my conclusion is that this limitation is not taught by the cited prior art.

(f) Limitation [1.b.v] "said data packs being portions of a complete data stream"

In subsubsection 3. I have provided my understanding of the term "data stream" and in subsubsection 4. I have provided my understanding of the term "complete data stream".

This limitation is not taught by Eylon. This is because Eylon constructed his own virtual file system, which allowed him to implement files as he saw fit, e.g., he implemented them without support for data streams. However, data streams were known to Eylon because NTFS in Windows had supported them since 1993. Thus, Eylon decided not to support them in his system.

The '808 Patent was constructed to be a file system filter driver, ordinarily resident on the system drive (e.g., the "C:" drive). Since Windows Vista, Microsoft has required that the system drive use the NTFS file system. Accordingly, the '808 Patent would see a need to support the features of the NTFS file system which included streams.

Further, the '808 Patent teaches a system in which the "complete data stream" may actually exist in multiple distinct local storage locations:

"Upon receiving a request for a data pack (aka 'asset'), the user mode application looks up one or more media locations from a list, as in step 322. Such list of locations (i.e., addresses) may be part of the reference table or a separate list, file or other reference data source. By way of example and not limitation, the asset may be available from an accessible web server 324, ftp server 326, a CD or DVD 328, nonvolatile memory (e.g., flash media) 33 0, hard drive 33 2, and any other source 334. If no such source is identified or accessible, then the user mode application looks for another updated list, as in step 336, and returns control to step 322. If the asset is available from

an accessible source, then a request to retrieve the asset is fulfilled, as in step 338, and receipt of the entire asset is confirmed, as in step 340. If any portion of the asset is unavailable from the targeted source, then control is passed to step 324 to identify another source. When the entire asset has been received, the user mode *copies the data pack into memory* or saves it to disk and replies back to the kernel mode, as in step 342." (Emphasis added, Ex. 1001Col. 7, Lines 1-19.)

Note that the data pack in this case may come from a source other than the underlying file itself. This is logical: if the data pack is available from an existing local location, there is no benefit to copying it into a second location.

Thus, the '808 Patent claims are distinguished from the prior art because it both offers support for data streams within file systems that support them, such as NTFS, and also because it allows the use of disjoint physical storage locations for logically associated data, as taught by the '808 Patent.

(g) Limitation [1.b.vi] "said I/O requests determining an order in which data packs are sought for the application,"

The prior art differs from this in two ways: first, the prior art teaches an active server that pushes content to the client and second, the prior art teaches a system in which the application's page faults are satisfied by the operating system's I/O requests.

Eylon teaches a system of predictive data push from the server: "Preferably, the server is configured to automatically forward sequences of streamlets to the client using a predictive 45 streaming engine which selects streamlets to forward according to dynamic statistical knowledge base generated by analyzing the various sequences in which the application program attempts to load itself into memory as various program features are accessed." (Ex. 1005, Col. 6, Lines 49) "In many cases, as a result of the predictive streaming, the desired page will already reside in the Cache/VFS 160." (Ex. 1005, Col. 10, Lines 47-49)

This differs from the teachings of the '808 Patent, in which a server need not be used, and if it is used is solely responsive: "Upon receiving a request for a data pack (aka 'asset'), the user mode application looks up one or more media locations from a list, as in step 322. Such list of locations

(i.e., addresses) may be part of the reference table or a separate list, file or other reference data source. By way of example and not 5 limitation, the asset may be available from an accessible web server 324, ftp server 326, a CD or DVD 328, nonvolatile memory (e.g., flash media) 33 0, hard drive 33 2, and any other source 334." (Ex. 1001, Col. 7, Lines 1-9). The '808 neither claims nor teaches that data is pushed from the source of the data to the client; it is only pulled by the client on-demand.

While the idea of pushing personal data from a client to the server is acceptable in the business environment that was the focus of Eylon, that is not necessarily acceptable in a larger environment, such as the streaming game delivery described in the '808 Patent. Eylon teaches us that the streaming client's information is stored and used on the streaming application server: "Preferably, the server is configured to automatically forward sequences of streamlets to the client using a predictive 45 streaming engine which selects streamlets to forward according to dynamic statistical knowledge base generated by analyzing the various sequences in which the application program attempts to load itself into memory as various program features are accessed. Such a knowledge base can 50 be generated by analyzing the past and present behavior of the current user, the behavior of the entire user group, or the behavior of subsets within that group. As a result, the streamlets 18 which are predicted to be needed at a given point during execution are automatically sent to the client 14 55 so that they are generally present before the application attempts to access them. Both code and data, including external files used by the application, can be predictively streamed in this manner." (Ex. 1005, Col. 6, Lines 44-58)

The '808 Patent has no centralized server to which detailed information about the client computer's usage of the applications. The '808 Patent's teachings that data can come from multiple sources means the usage data can be hidden by the computing device of the '808 Patent. This significant difference arises because of the streaming application server that is taught by Eylon, which is not taught, described, or required by the '808 Patent.

Eylon teaches: "Alternatively, details regarding the faulting input request issued by the operating system, such as the file name, offset, and length, can be forwarded to the server which then determines the appropriate streamlet data blocks to be returned and may also return additional

data not yet requested if it determines that the additional data be required shortly." (Ex. 1005, Col. 11, Lines 4-10) Eylon is teaching that the mapping of the page fault ("faulting input request") is mapped on the server side. There is no comparable teaching in the '808 Patent because there is no reliance upon the server.

The prior art teaches that it is page faults which drive the on-demand aspects of the streaming: "During normal operation of an application, the Windows operating system will periodically generate page faults and associated paging requests to load portions of the application into memory." (Ex. 1005, Col. 4, Lines 10-13) "As the loaded 10 executable or DLL is processed, the operating system issues a series of page faults or similar input requests to dynamically load additional needed portions of the file into memory." (Ex. 1005, Col. 10, Lines 9-12) "The additional parts will be loaded dynamically through faults conditions as functions are called from the DLL." (Ex. 1005, Col 10, Lines 24-25) "Turning to FIG. 6A, when a Page Fault exception occurs in the streaming application context, the operating system issues a paging request signaling that a specific page of code is needed for the streaming application." (Ex. 1005, Col. 10, Lines 34-37) "Alternatively, details regarding the faulting input request issued by the operating system, such as the file name, offset, and length, can be forwarded to the server which then determines the appropriate streamlet data blocks to be returned and may also return additional data not yet requested if it determines that the additional data be required shortly." (Ex. 1005, Col. 11, Lines 4-10) "When a page fault occurs, the application issues a data load which is passed in an appropriate format to the VFS manager 200 by the FSD driver 150." (Ex. 1005, Col 12, Lines 37-39) "As the header is processed by the application, a series of additional page faults associated with the needed library functions are issued." (Ex. 1005, Col. 12 Lines 46-49) "In particular, using the DLL header information, when the operating system detects that a unloaded function of a DLL is required, a data read operation is issued to load the needed portion of the DLL." (Ex. 1005, Col. 10, Lines 25-29)

Thus, Eylon is clear: it is not the application that drives this process, it is the streaming application server and the operating system on the client that drive it. This conflicts with the claims of the '808 Patent, which clearly states it is the application's demands that drive the streaming download process.

Comparing Eylon to the '808 Patent, it is insightful to note that the term "read" — which is the usual way of describing an application I/O operation that loads data from storage into application memory is used three times, one of which I quoted above, and is related to the program, not the program's data. Interestingly, the term "read" is used repeatedly throughout the '808 Patent (including figures, specification, and claims) focusing on the application's I/O operations that load data from storage into application memory. The '808 Patent never uses the term "fault" and instead uses the term "paging I/O" to refer to the process by which the operating system satisfies a page fault.

Thus, a POSITA reading Eylon would have no reason to consider the "read calls" of the application program because Eylon focuses on the "page fault" (or "fault") operations. Thus, the prior art is substantially different than the '808 Patent.

(k) Limitation [1.c.iv] "... said minifilter being configured to receive each I/O request from the application..."

I disagree that this is taught in the prior art.

The Petitioner asserts and Dr. Houh testifies that a POSITA would have been motivated to implement Eylon's streaming functionality, e.g., streaming support system 102, as a managed filter in Christiansen's system because a managed filter according to Christiansen provided "numerous advantages". However, Dr. Houh does not explain or justify his specific combination, providing just enough detail that one can "fill in the blanks" but without sufficient detail to understand the potential complications.

I would note that a POSITA, reading Christiansen, would consider the most logical combination of Eylon with Christiansen according to the teachings of Christiansen: "With contemporary operating systems, such as Microsoft Corporation's Windows® XP operating system with an underlying file system such as the Windows® NTFS (Windows® NT File System), FAT, CDFS, SMB redirector filesystem, or WebDav file systems, one or more file system filter drivers may be inserted between the I/O manager that receives user I/O requests and the file system driver." (Ex. 10060) In other words, the logical combination would be to place the mini-filter of Christiansen on top

of either the FSD, the VFS, or both, of Eylon. It seems likely that Dr. Houh has ignored this combination, taught by the prior art, because it does not lead to the system he needs to advance his arguments.

The Petitioner asserts and Dr. Houh testifies that a POSITA would have been motivated to implement Eylon's streaming functionality, e.g., streaming support system 102, as a managed filter in Christiansen's system because a managed filter according to Christiansen provided "numerous advantages." Dr. Houh's advantages are entirely the advantages of implementing a file system mini-filter rather than a file system filter driver. Dr. Houh fails to explain the advantages of implementing the FSD and VFS of Eylon as a file system filter driver. Thus, Dr. Houh's analysis fails to properly motivate his proposed combination.

In addition, file system filter drivers were commonly known in 1998 and thus they were known to Eylon. Dr. Houh does not point to any particular reason why Eylon's decision to implement a virtual file system was incorrect. He does not cite to any evidence that a legacy file system filter driver implementation of Eylon would have been a better solution. Without such an explanation, the balance of Dr. Houh's analysis is flawed because the rationale he provides requires that the system being converted to a mini-filter is already a legacy filter.

In my prior declaration I made a mistake: I tried to work through how I would create an embodiment of Eylon's functionality in the form of the mini-filter of Christiansen. I am not a POSITA. I was not a POSITA in 1996: I had been developing operating systems software, including networking software and file systems software since 1987. My first two peer reviewed academic papers were published in the early 1990s and they were about local and distributed file systems development, as listed in my CV. I started developing file systems on Windows NT in 1992, prior to its original release. I released sample source code for building file system filter drivers in 1995 and my company at the time started licensing a working framework for constructing file system filter drivers that addressed the issues for which Microsoft invented the filter manager (Ex. 2037). I built a framework to simplify the development of file systems on Windows that we commercially released in 1996 (Ex. 2083) and that toolkit is still available today (Ex. 2087). I started teaching others how to develop file sytems and file system filter drivers at week-long seminars in 1996 (Ex.

2065 shows the syllabus as of 2005). I taught Christiansen how Windows file systems worked in 1996, to augment his experience with UNIX and Novell file systems. Thus, I was not a POSITA in 1998 when Eylon did his work, nor was I in 2004 when Christiansen filed his application, nor in 2008 when the '808 Patent application was filed.

Thus, I have the skills and abilities to build the system that Dr. Houh should have clearly proposed: a file system mini-filter that provided the functionality of Eylon. I was actively working in the area of constructing isolation filters, which is the specific method needed to implement the system of Eylon as a file system filter or mini-filter driver, in the mid-2000s. It required inventing new terminology ("isolation filter") and concepts associated with that, which have since been adopted into the industry. A POSITA would not have been able to construct a file system filter or mini-filter that implemented the functionality of Eylon successfully.

However, the combination does not address this specific limitation. Recall the limitation says "*said minifilter being configured to receive each I/O request from the application*"

The language here is clear and unambiguous. It claims that the minifilter is configured to receive each I/O request *from the application*. This means: a mini-filter must be configured to receive application I/O. According to Microsoft's documentation, there are four kinds of I/O that can be received by a file system mini-filter (Ex. 2088 pp. 227-228): paging I/O, cached I/O, DASD I/O, and non-cached I/O. Direct Access Storage Device (DASD) I/O relates to I/O operations directed at the underlying disk drive and are not of interest in this discussion. The remaining three actually define **two** general types of I/O. Application I/O which may be cached or non-cached, and Paging I/O, which is always cached. Thus, if an I/O request indicates it is a Paging I/O, the file system or filter driver knows that the I/O was initiated by the operating system. **Only the operating system may initiate paging I/O**. All Paging I/O is non-cached; this is true on Windows, UNIX, and Linux, as I explain in more detail in § E.. Further, at least the Windows APIs for sending paging I/O operations are not available to application programs as I explain in more detail in § I..

Applications may, but typically do not, issue non-cached I/O. The restrictions on non-cached I/O are clearly documented across OS platforms. For example, I have included the Microsoft

documentation that indicates these restrictions: "This topic covers the various considerations for application control of file buffering, also known as unbuffered file input/output (I/O). File buffering is usually handled by the system behind the scenes and is considered part of file caching within the Windows operating system unless otherwise specified. Although the terms caching and buffering are sometimes used interchangeably, this topic uses the term buffering specifically in the context of explaining how to interact with data that is not being cached (buffered) by the system, where it is otherwise largely out of the direct control of user-mode applications. " (Ex. 2094). These restrictions are part of both user mode and kernel mode read and write I/O access (Ex. 2090, 2089, 2091, 2092, 2093, and 2088 pp. 744-746). Critically, the restrictions here are related to the underlying storage: "File access sizes, including the optional file offset in the OVERLAPPED structure, if specified, must be for a number of bytes that is an integer multiple of the volume sector size. For example, if the sector size is 512 bytes, an application can request reads and writes of 512, 1,024, 1,536, or 2,048 bytes, but not of 335, 981, or 7,171 bytes. File access buffer addresses for read and write operations should be physical sector-aligned, which means aligned on addresses in memory that are integer multiples of the volume's physical sector size. Depending on the disk, this requirement may not be enforced." (Ex. 2094) Indeed, these restrictions create substantial burden on application programmers as well as reduce the application performance in many cases. This is consistent with my statement earlier: cached I/O is the norm.

Since all paging I/O is also non-cached, it must adhere to these restrictions as well. Because page faults occur on page size (e.g., typically 4KB) boundaries and generate paging I/O operations in page size units (4KB) these restrictions do not impact paging I/O.

Thus, the limitation of the '808 Patent that the on-demand requester object is configured to receive application I/O is an important limitation that is not taught in the prior art — the prior art treats a 60 byte read as being equivalent to a 4KB read. While this is a much simpler model, it creates a read amplification issue. I observed this read amplification issue previously: "If a streaming application requires 200 unique assets in order to render a specific graphical page for the user, it is likely that it will need to access 200 different locations, across the set of files that correspond to the application. If the average size of these assets is 60 bytes, it would require

reading 12,000 bytes of data. If this data needs to be downloaded from the internet (as part of the "streaming on demand" application), this would generate 200 requests, one for each asset required. If the streaming on demand system were to "round up" to a 4KB boundary, each read would, on average, require 4036 extra bytes that are not useful in satisfying the current request. Thus, instead of requesting 12,000 bytes of data, the streaming on demand system using 4KB blocks would need 819200 bytes, or 807,200 extra bytes. In this example, that constitutes a 67x increase in the amount of data downloaded. Equally important, increasing the amount of extraneous data downloaded while performing on-demand streaming increases the amount of time required to receive it, process it, and can lead to delays in rendering. The slower the network, and higher the latency of these data transfer operations, the lower the quality of the gaming experience. While this is particularly important for interactive programs, it impacts all streaming on demand applications." (Paper No. 79)

Thus, we see key differences between the system of Eylon, the system of Christiansen, Dr. Houh's Eylon/Christiansen combination, and the claims of the '808 Patent: Eylon uses a server mediated streaming delivery system, implemented on the client as a virtual file system that supports compression and encryption of the streamlets, without any client-to-server connect. Christiansen uses data caching file system mini-filter over a local file system and a redirector (network file system) to store remote content retrieved from the redirector onto the local system. Dr. Houh asserts that his vague combination of Eylon and Christiansen supports all the features of Eylon, including encryption, compression, and sparseness. Dr. Houh does not explain how he will do these things; while they are possible, it took me — an expert in this field — more than a decade to do so. Dr. Houh ignores the conflicts between Eylon and Christiansen: Eylon requires an intelligent streaming application server, Christiansen relies upon a pre-existing network file system (or "redirector"). Does Dr. Houh's combined system abandon Eylon's teaching that the server pushes data to the client (Ex. 1005, Col. 6, Lines 59-62)? Does Dr. Houh's combined system implement the "no write back to the server" model that Eylon teaches (Ex. 1005, Col. 14, Lines 3-13) or the "write back to the server" model that Christiansen teaches (Ex. 1006, 2)? While Dr. Houh might

simply appeal to the remarkable skill set of his POSITA, as an expert I do not see any obvious "right answer" to these questions.

Even assuming Dr. Houh's POSITA's abilities to construct this system, it *still* does not teach a system that is entirely resident on the client. Thus, even giving the Petitioner every "benefit of the doubt" and allowing them to invoke *deus ex machina* multiple times in solving the problems of their combination it still falls short of teaching a system that matches the limitations of Claim 1. Nothing in the combination teaches an emphasis on the use of application I/O, instead teaching a system that focuses on improving the performance of the standard block streaming system using a server-driven mechanism.

More critical of Dr. Houh's proposed system, Dr. Houh's explanation on why someone would combine Eylon with Christiansen is flawed. First, Eylon teaches the construction of a *file system*, not a filter driver: "A set of streaming control modules are installed on the client system and include an intelligent caching system which is configured as a sparsely populated virtual file system ('VFS') which is accessed via a dedicated streaming file system ('FSD') driver." (Ex. 1005, Col 4, Lines 1-5).

Christiansen teaches the construction of a *mini-filter*: "In one implementation, filters comprise objects or the like that when instantiated register (e.g., during their initialization procedure) with a registration mechanism in the filter manager 220." (Ex. 1006, Para 34).

Ordinarily, the logical combination of a file system with a filter driver is to use the filter driver attached to the file system. In other words, the filter driver should be adding some feature or function that is not presently found on the target file system. Christiansen teaches the numerous advantages of file system filter drivers to augment or enhance the functionality of existing file systems. Nothing in Christiansen teaches *replacing* the file system with a file system filter driver.

Thus, Dr. Houh's combination of Eylon and Christiansen is itself not motivated by the ordinary way in which a file system and filter driver are combined. Instead, his combination seems to be focused on replacing the file system taught by Eylon with an implementation that borrows some elements of the file system mini-filter taught by Christiansen.

This non-obvious combination of Eylon and Christiansen places the burden on Dr. Houh to explain the rationale for his combination. To be persuasive, Dr. Houh's analysis must explain why it is advantageous to implement the file system (FSD and VFS) of Eylon as a file system filter driver; only then can he use the benefits of Christiansen's mini-filter over the file system filter implementation.

The advantages cited by Dr. Houh of a mini-filter implementation are specific to its implementation over a file system filter driver, **not** a file system driver. Indeed, the list of advantages Dr. Houh claims for the Eylon/Christiansen combination appear to largely consist of a recitation of the advantages of building a *file system mini-filter* versus a *legacy file system filter*. (Ex. 1010).

As I noted, however, this is a flawed analysis. Eylon would have known about the possibility of implementing his system as a file system filter driver. Rajeev Nagar's book (Ex. 2017) was published in 1997. I had published articles and sample code about developing file system filter drivers and my company licensed source code simplifying the implementation of file system filter drivers on Windows (Ex. 2037).

To rectify this flaw, Dr. Houh must show that implementing a file system as a file system filter would have been recognized as simpler or superior implementation model. I admit, I would be deeply skeptical of this because I have learned through my own direct experience, building filters with the functionality described by Eylon is not simpler, though it can provide a more robust implementation model — but these are the very benefits taught by the '808 Patent.

I have been developing file systems, legacy file systems, and file system mini-filters on Windows since 1992. It is my opinion that implementing the functionality of a file system *as a filter* is actually more complicated than implementing it as a file system. My opinion is not formed in the academic evaluation of materials provided to me by either party: it is a fundamental understanding forged in more than a decade of learning how to build filters with functionality akin to what would be needed to build the system of Eylon as a file system mini-filter driver. Because I have been teaching, writing, and explaining this for more than two decades, I have learned that the easiest way to help other understand why this is the case is to point to the greater complexity in the lower edge interface of a file system versus a file system mini-filter driver (Ex. 1006Figures 2-4,

6, 7, and 11 and 1005,Figures 4 and 7 and 1001Figure 2.) They both have the same upper-edge interface: the file systems API, which is quite complex (Ex. 2088 is almost 3,000 pages). The lower edge interface of a file system is much simpler; for media file systems, such as NTFS, it is three operations: read, write, and device control. Network file systems (redirectors) have a slightly more complex interface than the disk interface, but it is modeled closely on the BSD UNIX "sockets" model, which is well-understood and can largely be written and tested in user mode. Filters *can* be simple when they do not attempt to do much with the I/O operations passing through the filter. These are "passive" filters and Microsoft provides multiple examples of such filters. As a filter is called upon to do more complex processing, it must interact with the underlying file system and thus the developer must both understand how the underlying file system works and ensure that the filter's own activities do not "break" that functionality. This problem is a severe one: at one point, file system filters on Windows were the single most common cause of Windows operating systems crashes. To mitigate this, Microsoft has engaged in a multi-decade long process to add tools and mechanisms into Windows to simplify this, including week-long interoperability events ("PlugFest") which brings together Microsoft internal developers (including Christiansen, Pudipeddi and Thind) with external developers of file system mini-filters.

The Windows file systems API is "rich". Commensurately, this corresponds to a high degree of complexity, with rules and behaviors that are sometimes difficult to understand. For example, one of the important issues in this case is that the file systems API includes a read I/O handler. In fact, the file systems interface has multiple read I/O handlers: a "fast I/O" read handler, which is typically implemented by invoking a standard operating system provided handler that satisifes application I/O requests directly from the file system data cache and an `IRP_MJ_READ` handler for "normal" I/O operations. The `IRP_MJ_READ` handler itself handles cached I/O, which is the same type of I/O handled by the fast I/O handler, as well as paging I/O, which is initiated by the operating itself in response to a page fault, non-cached I/O, which included *all* paging I/O, and special I/O operations to support file servers (the `IRP_MN_MDL_READ` sub-operation) as well as a further specialized interface for reading data in compressed format from a compressed file. There

is a distinct/separate mechanism for reading the encrypted contents of files directly from a file where the data is already encrypted as well, but it uses `IRP_MJ_FILE_SYSTEM_CONTROL` instead.

Note that this example is just a *single* operation that a file system driver or file system legacy filter driver must be written to handle.

The lower edge of a file system driver is the storage API. For disks, this storage API consists of two primary operations: "read" and "write". These specify an offset and length, usually with alignment restrictions, but it is fairly simple. The FAT file system code, which Microsoft makes publicly available on Github.com, demonstrates a real-world implementation of the lower level storage interface. The network interface is slightly more complicated, but it is well-documented and can be used with the familiar API of the UNIX BSD sockets communications model.

A file system filter driver, including file system mini-filters have the same upper and lower edge interface: the upper is the same because these filters are processing the same requests that were originally handed to the file system driver. The lower edge interface is the same because filters and mini-filters interact with the file system and its interface is the file systems API interface. What makes building file system filters and mini-filters complex and challenging is that the interactions are subtle and complicated, including out-of order completion of operations, re-entrant page fault handling, and complex behaviors. For example, when an application program closes a file, the file system and any filters attached, including file system mini-filters, are notified via the `IRP_MJ_CLEANUP` call. When the Windows operating system closes the file, the file system and any filters attached, including file system mini-filters, are notified via the `IRP_MJ_CLOSE` call. One unexpected interaction here is that a mini-filter can be invoked in the `IRP_MJ_CLEANUP` handler and **before that operation is completed by the file system driver** a second call can be received by that mini-filter for the same file that is the `IRP_MJ_CLOSE` call. A filter must properly handle this situation. This situation arises because of the implementation of the file system. Thus, when a software developer implements a file system driver, that developer controls when this happens. When a software developer implements a file system *filter* or *mini-filter* driver, the behavior is dictated by how the given file system was implemented. Further, two file systems can (and do) implement this differently.

Thus, in my experience, a simple file system mini-filter is easy to implement. A file system mini-filter that wishes to implement any substantial amount of file systems logic becomes considerably more complicated than a file systems implementation. Thus, for at least the past two decades I have cautioned new developers entering this field to be cautious not to pick a file systems filter (and now mini-filter) implementation model because they think it is going to be simpler: in my experience, it is not.

Microsoft does promote the idea that implementing a file system mini-filter is easier than implementing a file system filter. For a simple file system filter, such as the examples they distribute, this is actually true. For a system trying to implement sparseness (Eylon) or security (Vinson), I have consistently and repeatedly found the opposite: it is more difficult to implement a file system filter or mini-filter than a file system driver. Thus, in my opinion, it is incorrect to assume that implementing the functionality of Eylon as a file system filter driver is a preferred embodiment.

Given this background on the differences in implementing file systems and file system filter drivers, I revisit Dr. Houh's list of benefits and explain why they are not persuasive (Ex. 1003, pp. 53-54):

- "a simpler, more reliable filter driver with less development effort" — this choice of wording clearly indicates a benefit of building a file system *mini* filter driver. Dr. Houh offers no analysis explaining why this filter driver analysis would apply to a file system. In my experience, the complexity of implementing file system and filter drivers vary substantially. The Petitioner's POSITA would know that this is the case.

- "dynamic load and unload, attach and detach of filters" — once again, Dr. Houh uses a benefit of using a mini-filter versus a legacy filter but does not provide any explanation of why this would apply to a file system driver. The language, however, is clear. It relates to these characteristics of *filters*. The Petitioner's POSITA would know that a file system driver can be dynamically loaded and unloaded (there are examples of this that Microsoft has published since at least the early 2000s) and that the Windows I/O subsystem would

automatically trigger the attachment and detachment of filters because that functionality had been present in Windows NT 3.1 (1993) through at least Windows Vista (2006).

- "the ability to attach filters at a well-defined location in the filter stack" — once again, Dr. Houh already assumes that the correct implementation of Eylon on Windows would be to implement it as a file system filter driver. This "benefit" would not apply to a file system driver because it is not a filter and its location is *already* at a well-defined location in the I/O stack, notably below all the file system filter drivers, including file system mini-filter drivers. A POSITA using the Petitioner's definition would know that a file system driver is unconcerned about the location of filters in the stack.

- "better context management: fast, clean, reliable contexts for file objects, streams, files, instances, and volumes" — these file system mini-filter context structures are implemented by layering additional state on top of existing file system data structures. Thus, an implementation of the system of Eylon on Windows as a file system driver would not gain access to these things because it will have already defined them and be providing them to filter manger, which in turn exports them to file system mini-filters. A POSITA would know that a file system driver already defines context structures for file objects, streams, files, instances, *and* volumes: "The **FsRtlSetupAdvancedHeader** macro is used by file systems to initialize an **FSRTL_ADVANCED_FCB_HEADER** structure for use with filter contexts." (Ex. 2088 pp. 1598-1600). Thus, this purported advantage is not applicable to a file system driver. (Ex. 2088 for descriptions of this function and data structure)

- "a host of utility routines including support for looking up names and caching them for efficient access, communication between minifilters and user mode services, and IO queuing." — Dr. Houh describes issues that, once again, are benefits of implementing a *mini-filter* versus a *legacy filter*. These are not targeted to the file system itself. Further, I would note that by 2008 the name management functions were being used by drivers completely

unrelated to file system filter drivers, such as the network firewall, and thus it was well known that many of these functions could be used from any driver, not merely a file system mini-filter driver. A POSITA would know that a file system driver can use the existing mechanisms for communications with user mode services, and I/O queueing, both of which existed prior to the introduction of filter manager in Windows. The naming support routines provided by Filter Manager can be called from any driver, they do not require building a file system filter driver. For example, the function `FltParseFileInformation` can be invoked by an arbitrary kernel mode driver in Windows, it does not require that such a driver be a file system mini-filter driver: "FltParseFileNameInformation parses the contents of a `FLT_FILE_NAME_INFORMATION` structure." (Ex. 2088 pp. 687-689also Ex. 2088 for a partial listing of some documented APIs and data structures used for file systems and file system mini-filter drivers).

- "support for non-recursive I/O so managed filter generated I/O can easily be seen only by lower filters and the file system." — Dr. Houh's own language here clearly expresses that this applies to the filter layer. I would note, however, that a POSITA would have already known about the `IoCreateFileSpecifyDeviceObjectHint` (Ex. 2088 pp. 1165-1176) which is a general purpose device driver I/O operation that is also used by filter manager to implement this functionality for file system mini-filters. A file system driver could use these functions directly, but would not need to do so, as a file system driver can also use functions for directly targeting I/O operations, such as `IoAllocateIrpEx` (Ex. 2049) and `IoBuildAsynchronousFsdRequest` (Ex. 2050), both of which existed prior to filter manager and are used in the sample file systems that Microsoft was distributing prior to 2008. Thus, this issue of re-entrancy is only an issue for file system filters and mini-filters, not file system drivers.

As I noted previously, Dr. Houh asserts that Eylon should be implemented as a file system filter driver, without providing us with any explanation as to why. Thus, Dr. Houh has constructed a

"straw man" that he uses to extoll the virtues of converting from the legacy filter solution that he has chosen to the file system mini-filter solution taught by Christiansen.

As I noted previously, Dr. Houh fails to address the areas of functionality that conflict between Eylon and Christiansen. This leaves the Board to assume that Microsoft's remarkable POSITA chooses exactly the right combination of trade-offs to map into teachings that cover the '808 Patent. Even given the work of this truly remarkable POSITA, the magical combination of Eylon and Christiansen that Dr. Houh teaches still fails in two clear ways: (1) it fails to teach a client-only system, since it must at least have some sort of remote server that provides the streaming data content. This does not teach local media usage, peer-to-peer usage, and the novel combinations of them (e.g., a game player purchases a copy on optical media and then the '808 based system pulls content from that media, older versions of the game assets that were downloaded to local storage, other computer peers using the same game content, as well as standard web servers); and (2) it fails to teach a system in which data modifications are written back to the local storage *but not* written to the remote storage. Eylon teaches that changes are not written back and Christiansen teaches that changes are propagated back to the file server. Neither teaches they are written only to local storage. While I have pointed out other differences (e.g., the application I/O versus paging I/O issue,) these two distinctions are sufficient to demonstrate that, even given the remarkable insight of the Petitioner's POSITA, the combined system still fails to teach the claims of the '808 Patent.

Dr. Houh does not provide any evidence supporting for his implicit assumption that the Petitioner's remarkably astute POSITA would have thought that the correct way to implement the teachings of Eylon was to implement it as a file system filter driver. Note that it *is* a teaching of the '808 Patent: "An important aspect of a process according to principles of the invention is that it does not require any virtual drive, device or the like. It works with the original hard drive and disk partition. Many application streaming systems and processes that currently exist facilitate streaming by using a virtual drive, file system driver or the like which are difficult to implement in compliance with security protocols and inefficient to maintain. In contrast, the Subject invention

uniquely uses a mini-filter, also known as a file system filter driver, i.e., an object that attaches to a file system driver and filters requests only." (Ex. 1001, Col. 10 Line 63 to Col. 11 Line 5)

There are other reasons why Dr. Houh's combined system does not make sense:

- Sparse storage support — Eylon teaches the benefits of sparse local storage (Ex. 1005, Col 4, Line 3). The mini-filter Dr. Houh proposes is now faced with a complication: the NTFS file system supports sparse files but the FAT file systems does not (Ex. 2051, p. 27). A sparse file is one in which regions of the file have no corresponding storage associated with them. Sparse files are not unique to NTFS, but they are not supported by the FAT File system. In 2008 Dr. Houh's mini-filter would have needed to work with the FAT file system because FAT was still supported for the system drive on Windows XP. It is difficult for me to imagine that the Petitioner's remarkably astute POSITA would overlook this limitation.

- Loss of Cache Control — Eylon implements a virtual file system. As such, Eylon is a file system and maintains control over the file system data cache for its files. For example, Eylon's file system can use `CcCoherencyFlushAndPurgeCache`, a function that permits forcing data stored in the file system data cache (Ex. 2088 pp 1378-1379). This is just *one* of the more than three dozen cache manager functions that the Petitioner documents. As a general rule, they can be used by the driver that owns the cache which is ordinarily the file system (e.g., NTFS or FAT). Eylon's system controls the cache. Dr. Houh's system does not control the cache.

- Encrypted and Proprietary data format — "To prevent unauthorized access to the streamlets (and thus various portions of the application files), one or more of the streamlets in the library, the streamlet map, and the application file structure can be stored in an encrypted or other 55 proprietary format." (Ex. 1005,Col. 11, Lines 52-56) Supporting this proprietary encryption and data format is quite straight-forward with the Eylon-controlled VFS but is "left as an exercise for the reader" in the combination. I do know, however, based upon my decades of experience building encryption and compression services as file system filter

drivers on Windows that the level of effort required for this exercise is significantly higher than the effort required to construct Eylon's system as originally specified. This is largely driven by the fact that a filter supporting encryption and proprietary data formats must control its cache, which is the standard for a file system driver (as I noted above) but is not the standard for a file system mini-filter driver. Indeed, the need to support encryption and proprietary data formats was my motivation for inventing and building isolation filters.

- I/O amplification — I previously explained the read amplification that arises in a block-based system. I have also noted the "loss of control" that a filter has versus a file system with respect to the cache. These two factors are exacerbated in Dr. Houh's mini-filter because when Dr. Houh's mini-filter is layered over the top of NTFS, the page fault granularity behavior is now dictated by NTFS. Prior to Windows Vista, Microsoft would not issue a paging I/O larger than 64KB in a single operation. Beginning with Windows Vista, and greatly improved in more recent versions, Microsoft has increased this size and it is now common to observe paging I/O operations that are up to 32MB in size. Thus, the 60 byte read that I previously observed might swell to a 4KB read can now swell to a 32MB read. Eylon, as a file system drive, had control over this and even if he had not exercised this control, he would have used the default behavior, which was 4KB page fault clustering. The specific call that controls this is `CcSetReadAheadGranularity` (Ex. 2088 pp 1457-1458). This is just one of the numerous "cache manager" calls that can only safely be made by the owner of the file system data cache, which is ordinarily the file system driver. Indeed, the work that I have done to realize systems like Dr. Houh's mini-filter work by managing the data cache independent of the underlying file systems. In such systems, we ordinarily only perform non-cached I/O to files that are being managed by the filter, so the file is only cached once — in the cache we control. This is a complex and sophisticated technique, an understanding of which the Petitioner has not yet attributed to their already remarkable POSITA.

- Modifying data contents — the system of Eylon can store its meta-data in any way that it chooses to because it controls the allocation and layout of information within the file. Dr. Houh's system stores location information in the file: "[w]hile Eylon does not specify precisely where its system stores the server address, a Skilled Artisan would find the application file structure to be a logical and efficient place to store the server address, as the application file structure is designed to store the location of files." (Ex. 100345) The idea of "adding information to the file" from a file system filter driver is hardly a new one. The Petitioner's POSITA would be well aware that one can not just "add data to a file" and expect the file to work correctly. If this is in doubt, I invite the read to open the the Petitioner word executable in Notepad nd insert the location of that executable at the front of the file. It will not execute. I know this because the idea of adding information into the file is one that I have implemented: *any* change to the "application file structure" that is visible to the application is likely to interfere with the correct behavior of the application and damage to the meaning of that data. Of course the degree to which the file is damaged varies by application: adding text to the front of a text file does not usually lead to data loss. Adding text to the front of a video or photo file is likely to make the data unusable. This was my motivation for creating the isolation filter: it can embed the data within the file, but makes it invisible to the applications accessing the file by ensuring the data is not loaded into the file system data cache.

These issues that arise from attempting to implement the functionality of Eylon in Dr. Houh's Christiansen inspired minifilter are not addressed by Dr. Houh. These issue literally took me *years* to overcome in a reliable and robust fashion.

Thus, the Petitioner's POSITA, knowing these serious limitations of trying to implement the data management functionality of a file system driver inside a file system mini-filter would not have found Dr. Houh's novel combination of Eylon with Christiansen persuasive. Given that the combination is not obvious to the Petitioner's POSITA, it does not demonstrate that the '808 Patent is obvious over Eylon and Christiansen.

To further support my contention that the functionality of file systems is not ordinarily implemented in file system filter driver, except when necessary, I observe that Microsoft has implemented multiple file systems since 1998 and none of them have been done as file system filter drivers, even though that is a possible option for each of them. This includes ExFAT, CLFS, and ReFS, all which have been developed since 1998. Also, Microsoft's network redirectors for SMB2 and SMB3 have been implemented as network file systems, not as file system filters, even though such a file system filter would be closer to the teachings of Christiansen. (Ex. 2053) Given that this is core to how Windows works, it is not surprising that Microsoft exhibits considerable understanding of these technologies. (Ex. 2054, 2055, and 2056)

Microsoft, despite its expertise in building file systems and file system filter drivers, decided not to implement any of these file systems as filter drivers. Indeed, the emergence of filter drivers with functionality comparable to the '808 Patent all occur within the past six years, which is well after the '808 patent was issued.

Dr. Houh's combination of Eylon and Christiansen leads him to a minifilter combination that he maintains is obvious to the Petitioner's POSITA. However, he fails to explain how his minifilter is better than other radically different implementation alternatives. The Petitioner's POSITA would have at least considered these alternative strategies, yet Dr. Houh does not explain to us why his minifilter is obviously superior to these other alternatives:

- The system of Eylon was already specified as being a file system driver and VFS. Thus, there is a motivation to implement it as specified to minimize the development time and effort required. By 2008 Microsoft provided extensive documentation about the construction of file systems on Windows, as well as examples of various file systems. A POSITA, using the Petitioner's definition, would have known of such documentation and examples and be motivated to keep the implementation simple and conform to the existing specification as closely as possible to minimize development and business risk.

- Prior to 2008 one could have implemented the system of Eylon by combining the existing VFS implementation of Eylon with the OSR File System Development Kit, which was a com-

mercially available toolkit. By 2008, Microsoft itself was commercially distributing software that used the OSR FSDK, specifically for providing streaming applications. I describe the OSR FSDK in more detail in subsection M.. The benefits of this would have been to use a single source code base between UNIX, Linux, and Windows systems and to decrease the development time; OSR was able to port UNIX file systems and have functioning versions within a matter of weeks. A POSITA would appreciate these benefits and the corresponding decrease development risk and the business benefits of being able to offer a product to market more quickly.

- Prior to 2008 one could have implemented the system of Eylon by combining the VFS of Eylon with a file system mini-filter driver that implemented a name space provider (Ex. 2088 pp. 890-892). Such a system would have allowed the file system mini-filter benefits taught by the '808 patent (Ex. 1001, Col 10 Line 62 to Col 11 Line 19) without requiring implementing the VFS of Eylon as a file system mini-filter. Microsoft clearly documented this approach as well (Ex. 2088 pp. 890-892). Such a system would have been simpler to implement than the proposed system of Dr. Houh because it would have only required implementing the namespace portion of the file system mini-filter, rather than the I/O handling portions of the file system mini-filter. One of the filter manager developers posted about name space providers; the post is from 2011 but the information described was generally known in 2008 (Ex. 2057).

- Prior to 2008 one could have implemented the system of Eylon as a file system mini-redirector. Such a file system would have used the "rdbss" mini-redirector driver inside the Windows kernel to present the system of Eylon as a network file system, which would have made it like other existing Microsoft-provided files. Further, Windows provides an entire caching layer for network file systems, that would have allowed the implementation of Eylon while leveraging that existing cache logic. A POSITA would appreciate the benefits inherent in using the data

caching layer that Microsoft provides for remote file systems and the decreased development risk. I provide more information about network mini-redirectors in subsection N..

- Prior to 2008 one could have implemented the system of Eylon without developing any kernel mode software. ELDOS, a commercial company that offered a variety of technology toolkits, offered its "callback file system", which permitted development of a Windows file system without implementing any kernel mode code. A POSITA, using either definition, would appreciate that avoiding kernel mode programming significantly decreases development risk and offers business benefits because the time to market is much shorter (Ex. 2058).

- Prior to 2008 one could have implemented similar functionality by building a "shell extension" for Windows Explorer, which is the graphical component of Windows that shows file system resources. An example of such a shell extension is how certain types of archive files can be trivially expanded and viewed directly. For example, a "zip" archive can be opened and browsed as if it were part of the file system on Windows (Ex. 2062). Similarly the contents of an ISO file system image, such as used by CD and DVD based storage devices, can be "mounted" and perused, even though it is only a local file (Ex. 2061). A third example is a virtual disk drive, such as a "Virtual Hard Drive" that Microsoft supported for Windows before 2008 (Ex. 2059). A virtual hard disk is simply a file stored within an existing file system that can then be mounted as if it were a distinct file system. This "mount" operation can be automated using a shell extension so simply double clicking on the VHD causes the mount operation to occur. The system of Eylon could be realized on Windows by constructing a shell extension that presents its own namespace (Ex. 2060). None of these shell extension based techniques would require constructing a file system or a file system filter driver. Further, a POSITA would appreciate that such a system can be implemented without requiring the development of *any* kernel mode software.

Dr. Houh concludes it makes more sense to construct the file system of Eylon as a file system filter driver on Windows, eschewing Eylon's decision not to do so in, even though it would have

been a known option to Eylon. Dr. Houh does not explain why a POSITA would override Eylon's decision to put the key functionality, such as the pre-fetching logic, on the server, since the '808 patent claims a system where the entire process is driven by the client computer (the "computing device" of Claims 1 and 14). Having overridden the teachings of Eylon, Dr. Houh then goes on to conclude that it only makes sense to implement his novel client-only filter based version of Eylon as a file system mini-filter driver. Conveniently, this fits Microsoft's messaging since they released filter manager.

Thus, Dr. Houh's analysis is critically flawed: the proposed combination is not obvious and in fact will either be more difficult to implement as a file system mini-filter than it was as a virtual file system or it will sacrifice extensive functionality.

In fact, it is my opinion that a POSITA would conclude that the safest proposition to implement a non-VFS based system comparable to Eylon's on Windows would have been to construct a shell extension. This conclusion would have been obvious because a shell extension does not require any kernel programming. The skills required to construct a shell extension are commonly available because there are far more Windows applications, many of which already use shell extensions, and thus it would be easy to find such skilled software developers. The skills required to construct a Windows kernel mode device driver are much rarer — these are the skills of Mr. Caitlin, the Petitioner's other cited expert. The number of software developers with the skills required to construct a Windows file system driver, file system filter driver, or file systems mini-filter driver are rare relative to the number of software developers with the skills needed to develop a Windows kernel mode device driver. Further, Microsoft's decision to implement specialized driver technologies, such as the *Kernel Mode Driver Framework* — KMDF — (Ex. 2063) and the *User Mode Driver Framework* — UMDF — (Ex 2064). further minimized the need for software developers to obtain the specialized skills required to build a Windows file system, file system filter driver, or file system mini-filter driver. I note that Microsoft's WDF models (KMDF and UMDF) do not support file systems, file system filter drivers, or file system mini-filter driver development. Microsoft did not even have user mode file system filter libraries until the introduction of the Cloud

Filter APIs in Windows 10. Thus, it is my professional opinion that the skills required to develop file systems, file system filters, and file system mini-filters were uncommon in 2008.

As evidence that knowledge about developing file system drivers, file system filter drivers, and file system mini-filter drivers was more specific than knowing how to develop file system drivers, I note that the courses that I taught on constructing file systems and file system filters prior to 2008 explicitly listed knowing how to build Windows Kernel Mode Device Drivers as a pre-requisite for the course (Ex. 2065). I would also note that the course I taught on constructing file system mini-filters in 2007 explicitly required that a student have taken my course "Developing File Systems for Windows" (Ex. 2065). This is consistent with my observation that file systems — including file system filter driver — development experience was a rare skill set. Another useful metric for understanding the complexity is that the printed copy of Microsoft's Installable file systems documentation (Ex. 2088) is now 2909 pages and *that* assumes the author is familiar with the Windows NT device driver model (Ex. 2084). This is simply the *reference* documentation to the interface. It does not include architecture and design documentation, or the samples that Microsoft distributes.

Thus, using a shell extension to embody the teaching of Eylon would have used a more broadly known technology, with lower risk, and available to a much larger pool of skilled software developers than a file system, file system filter, or file system mini-filter development project. The relative scarcity of software developers with Windows kernel mode programming also meant that software developers with the requisite skills were rare and more costly than software developers familiar with developing Windows kernel mode file system mini-filter drivers.

A shell extension based solution does have limitations; for some companies those limitations might not have been acceptable, in which case they would look to a more general solution. In a case where the limitations of a shell extension implementation are not acceptable, a POSITA, using either definition, would reasonably conclude that the least risky proposition to implement the system of Eylon on Windows as a kernel mode file system would have been to build it as a user mode service with the ELDOS toolkit. The option of implementing it using the OSR FSDK would be a good option when porting an existing UNIX VFS based implementation of Eylon to

Windows. The option of implementing it as a network mini-redirector would have been yet another option, since Microsoft supported such file systems and it would have provided benefits for caching that would have greatly simplified the implementation as the Eylon-mini-redirector would not have needed to manage cache storage at all. The option of implementing it as a file system driver on Windows would have been a viable option as well.

I am an expert in building file systems and file system filter drivers, particularly on the Windows platform. Based upon my own experience in implementing a file system mini-filter on Windows embodying the teachings of the '808 Patent, I estimate that the primary complexity of such a file system mini-filter is related to managing the integrated file system data cache in Windows. For the mini-filter to be able to implement the specific teachings of the '808 Patent — specifically "... and flush the kernel buffer out of memory." (Ex. 1001, Col 12, Lines 14-15). This can only be done by a mini-filter if it controls that kernel buffer; since the buffer is in the cache, the file system mini-filter must also own the cache. Conceptually, this is simple, but from an implementation standpoint it is extremely difficult and challenging to do. Once done, achieving the described functionality is simply a matter of calling **CcCoherencyFlushAndPurgeCache**. This function call was added in Windows 7, which was released in October, 2009, but had been announced two years earlier by Microsoft at their File Systems "Plug Fest"; prior to Windows 7 there were other more complex mechanisms that a file system driver could use to force purging the file system data cache controlled by a file system driver. What was not — and is still not — an option for a file system filter or mini-filter, is to purge the file system data cache of an underlying file system. I did observe a number of parties attempting to do so, usually because they would ask questions about this when their customers experienced operating system crashes ("blue screens of death") because of the fundamentally unsafe nature of these operations. In 2009 I publicly posted about this function and clearly identified that only a **file system** that owns the cache and is holding the proper locks can do this (Ex. 2088 pp. 1378-1379).

A person of sufficient skill to implement a file system mini-filter that implemented integration with the file system data cache would have had sufficient skill to implement a file system capable of implementing the system of Eylon. Further, a person of sufficient skill to implement a file system

mini-filter that implemented integration with the file system data cache would have realized that the benefits of implementing a *file system filter driver* as a *mini-filter* did not apply to a *file system driver* that implemented a Virtual File System as taught by Eylon.

(l) Limitation [1.c.v] "said minifilter being further configured to reference a table that includes names and paths with at least one offset address and length where each data pack required to fulfill each I/O request is located"

The Petitioner asserts that this limitation is taught by their prior art references. While superficially this appears to be the case, as I have noted previously, it assumes accepting a construction for their term "streamlet" that requires a POSITA with the remarkable insight to realize that exploiting the knowledge of the application's data use in forming variable sized blocks was a teaching of Eylon even though the text and explanatory diagrams do not teach this.

Specifically, I would note that Eylon Figure 7 provides a graphical explanation of how Eylon envisioned this working. This exemplary table shows us a file name, an offset, and a "streamlet ID". The path is not included, though I agree that one could construct such a structure though I do not make light of this because while conceptually simple there is complexity in building a system that can support the 32,767 character long path names that Windows supports. More interesting is the lack of a length value in Figure 7. Eylon's suggested implementation model of using a bitmap: "In one implementation, the map is stored, at least in part, as a binary table with a bit corresponding to each block in each file." (Ex. 1005, Col. 12, Lines 25-27) Such an implementation would only work if the blocks are fixed size. Thus, while one can, in hindsight, argue that Eylon *could* have supported variable size streamlets, he provides no motivation to do so, no description that suggests he envisioned doing so, and no preference to structures that would have been required.

Eylon teaches a traditional hierarchical file system: directories containing other directories and files, with files consisting of a set of fixed size allocation units (blocks) that may be sparsely allocated: "For example, the code page starting at an offset of 4 k in the DLL file is not defined in the map, here having an associated streamlet ID of "x", and is therefore considered absent."

(Ex. 1005, Col 12, Lines 18-21) Had Eylon considered that variable sized regions were useful, this would have been an ideal place to demonstrate this: by showing that the "absent" region started at 12K with a length of 40K.

This is quite different than the teachings in the '808 Patent:"Counted assets are assets that have been tagged in a particular order for transfer, and may also refer to other counted assets to provide a dynamic loading tree that may change as the user operates the application. Common assets are objects that have been shared between multiple counted segments. In many cases, application operation will reuse the same sound, graphic, code or other resource in multiple instances. This allows for the media to be streamed with a counted object, but not be required to be streamed again from any medium." (Ex. 1001, Col 10., lines 27-36)

There is nothing in Eylon that teaches the reference table as it is explained in the '808 Patent: it isn't dynamic, it only captures the idea that elements are duplicated when they are "absent". There is no concept of deduplicating identical streamlets. That also makes sense to me, because in Eylon's fixed-block system such asset reuse would not correspond to identical blocks/streamlets. Since the likelihood that two blocks in Eylon's system are identical, other than the absent blocks, is low, there is no reason Eylon would have considered it beneficial to identify duplicates.

Thus, the streamlet map of Eylon is notably different than the dynamic asset map taught by the '808 Patent. Thus, the reference table claimed by the '808 Patent is clearly different than the streamlet map of Eylon. Similarly, there is nothing in Christiansen that teaches us the '808's novel dynamic reference table.

Even in combination, the teachings of Eylon and Christiansen fall short of the intent of the claims of the '808 Patent, as viewed in terms of the specification of the '808 Patent.

I would also note that in Dr. Houh's minifilter constructed by combining Eylon and Christiansen, he has created considerable work for himself. As I observed in my analysis for the previous limitation (1.c.iv) the elimination of control over the underlying storage complicates matters. His VFS manager no longer owns the storage and thus must rely upon the underlying file system to maintain the application file structure; he can no longer store the location information for the files within the application file structure because he no longer controls the contents of said files. He

must store the streamlet map and streamlet library into the storage area of the underlying file system, which means it is either now visible to other applications or his filter must block access to it.

He can take over control of some, or all, of these pieces of functionality, but this creates considerable additional work. Without a very persuasive reason to implement it in this way, the complexity of this system is motivation *not* to implement the novel combination of Eylon and Christiansen that Dr. Houh's minifilter represents.

(m) Limitation [1.c.vi] "... said minifilter being further configured to determine if the data pack has been streamed to the system, and..."

The '808 specification provides a specific method of grouping the data packs into three specific types that is not taught in the prior art: "Counted assets are assets that have been tagged in a particular order for transfer, and may also refer to other counted assets to provide a dynamic loading tree that may change as the user operates the application. Common assets are objects that have been shared between multiple counted segments. In many cases, application operation will reuse the same sound, graphic, code or other resource in multiple instances. This allows for the media to be streamed with a counted object, but not be required to be streamed again from any medium." (Ex. 1001, Col 10., lines 27-36)

One reason why this claim limitation is not obvious over the prior art is that it captures the '808 Patent's teaching that a system which uses the application's preferred I/O size yields better user-perceived performance. The prior art systems also keep track of what has been streamed to the system, but they do so in terms of blocks, which are typically not cognizant of the structure of the application's data within the blocks.

A second reason why this claim limitation is not obvious over the prior art is that the '808 Patent teaches that the content can be loaded from local sources. While the term used here is "streamed" there is nothing in the prior art that teaches using a pre-existing local source for fetching the data. Thus, the term "streamed" here is distinct from the way it is used in the prior art. (Ex. 1001, Col. 8-9).

(n) Limitation [1.d] "said application being a program configured to use the data packs that fulfill each I/O request when the data packs are available on the computing device."

This limitation is not taught in the prior art. The Petitioner has erred in arguing that it has, likely because Eylon never discusses application initiated I/O and thus the Petitioner does not recognize the distinction between the operating system initiated I/O taught by Eylon and the application initiated I/O taught by the '808 Patent.

Eylon uses the term "read" five times in the specification: "When the streamlets are returned, they are stored in the VFS and the read or paging request from the operating system is satisfied." (Ex. 1005, Col. 4, Lines 28-30) "For example, in a Windows operating environment, the LoadLibrary API can be called from an application to have the operating system load a DLL. In response, the header of the DLL is read and loaded into memory." (Ex. 1005, Col. 4, Lines 16-19) "In particular, using the DLL header information, when the operating system detects that a unloaded function of a DLL is required, a data read operation is issued to load the needed portion of the DLL." (Ex. 1005, Col. 4, Lines 25-29) "Such schemes can are also be substantially slower, since they do not take advantage of "deep" predictive algorithms, but can only use simple, heuristic approaches such as cache read-ahead, which deliver several contiguous blocks of data into the local Client context in addition to the required block(s)." (Ex. 1005, Col. 14, Lines 20-25) "To prevent a user from copying the stored streamlets to a local drive, the streaming FSD 150 can be configured to only process read or page-in requests from the operating system which are generated on behalf of the application itself or on behalf of other approved processes." (Ex. 1005, Col. 13, Lines 61-66)

Of these uses of the term "read" only two relate to an application using the read call to load data into application memory from a persistent storage device and in both of those cases it is combined with paging: "the read or paging request from the operating system" and "process read or page-in requests from the operating system." Note that in *both* of these cases Eylon teaches that the read comes from the operating system, which is consistent with the ordinary behavior of cached I/O and its integration with the file system data cache (Windows) or page/buffer cache (UNIX/Linux). This is consistent with Eylon's use of the term "fault" or "page fault" throughout

the specification, since data is loaded into the file system data cache using the normal OS page fault mechanism.

In comparison, the '808 Patent uses the term "read" extensively to describe application I/O. (Ex. 1001, Figure 3A, 3B, 4, Col. 2, Lines 59 and 61, Col. 3, Lines 11, 31, 33, Col. 4, Line 49, Col. 6, Lines 19, 40, 43, 55, 58, 60, and 64, Col. 7 Lines 25, 29, 50, 57, Col. 8, Lines 3, 7, 37, 41, Col. 9, Line 33, Col. 10, Lines 52, 57, Col. 12, Lines 4, 12) It distinguishes "paging I/O" from "application I/O", which is consistent with the ordinary use of these terms and are familiar to a POSITA. The '808 Patent teaches that I/O may originate from something other than the operating system, which is different than the teachings of Eylon.

Eylon does not teach about application reads, only operating system initated reads. The '808 Patent teaches us that I/O from the application can be directly handled, not just operating system initiated I/O. This distinction is important because it is the application I/O that the '808 Patent teaches that is used to drive the on-demand load of content needed by the application.

(o) Alleged Reasons for Combining the Teachings of the References The Petitioner maintains that it is reasonable to combine the system to realize the file system minifilter presented by Dr. Houh. However, as I have previously observed (see my analysis for Limitation 1.c.iv) the simplest combination of Eylon with Christiansen is the one taught by Christiansen: to layer the minifilter of Christiansen over the file system of Eylon.

This combination does not negate the '808 Patent's claims over the prior art because the prior art teachings are based on a virtual file system. In addition, the obvious combination of these prior art references, taught by the prior art cited, as well as Pudipeddi, which the Petitioner cites elsewhere (Ex. 1007) does not provide any benefits over Eylon.

Dr. Houh's novel combination, in which Eylon is transformed first from a file system driver then to a file system filter driver and then, using the teachings of Christiansen, into a file system minifilter driver does get him closer to what the Petitioner needs. Unfortunately, even *this* creative minifilter fails to teach the claims of the '808 Patent at least because: (1) Dr. Houh's system still relies upon the push based custom streaming applicaiton server of Eylon; and (2) while Dr.

Houh's system fails to resolve the conflicts between the teachings of Eylon and the teachings of Christiansen, there is no resolution that teaches the limitations of the '808 Patent.

Beyond these basic reasons, which I describe more fully in my analysis of limtiation 1.c.iv, there is also the fundamental observation that Dr. Houh's system is far more complex than he admits in his declaration; attempt to mitigate that complexity require compromises that yield a less useful system.

In addition, I have previously explained that there are clearly other implementation models, including models that involve no kernel mode programming to implement the system of Eylon. It would be clear to a POSITA that, barring a compelling reason to implement software inside the Windows operating system, it is better to use an entirely user-mode solution because it minimizes development and business risks. If the functionality of a user-mode only solution is insufficient to meet business needs, using existing off-the shelf components to implement the system make the moss sense (e.g., the ELDOS system). A thorough analysis would have explained all these options to the Board.

The Petitioner's POSITA would be aware of the broad range of implementation models and choose one that best meets their needs. Based upon my analysis, it seems clear that a POSITA would choose either one of the user mode only solutions or the ELDOS solution because none of those involve *any* kernel mode programming experience; this greatly expands the pool of implementation talent that can build the system of EYLON. None of these solutions require implementing a file system filter or file system mini-filter. Of the options that require kernel mode Windows programming, each has benefits and disadvantages. I do agree that *if* one decided to implement a system like Eylon as a filter, it should be done as a mini-filter.

Finally, I will note that the conversion of a working file system to a working file system mini-filter driver is not a trivial undertaking. As I previously noted, Microsoft used the OSR FSDK as part of their App-V application virtualization product by way of their acquisition of Softricity in 2006. (Ex. 2095 and 2096) I was the designer and heavily involved in the implementation and support of the FSDK. From the time Microsoft acquired Softricity in 2006 Microsoft advised us they were in the process of converting their file system, which is used for streaming applications,

into a file system mini-filter driver. They discontinued paying OSR maintenance and support in 2015. Thus, it required almost a decade for them to convert from a file system *driver* to a file system *filter*.

(p) Conclusion for Claim 1 For the reasons that I have stated in earlier sections, I disagree with the conclusions of the Board regarding Claim 1 because:

- Limitation 1.b.ii is not taught by Eylon or Christiansen, which deals with paging I/O only, not application I/O. I have explained why a system that only uses paging I/O, such as is clearly described within Eylon, can be implemented without ever interacting with application I/O. I have also clarified that paging I/O cannot be initiated directly by an application and that paging I/O cannot reliably be associated with a specific application.

- Limitation 1.b.iii is not taught by Eylon or Christiansen because neither Eylon nor Christiansen teach handling application I/O because there is no motivation for them to do so — handling only paging I/O is sufficient to be correct, it is consistent with the fixed block scheme of Eylon and can be made to work even if there is a mismatch between the block size of the streaming application server and the client on which the application executed. Because there is no added benefit to handling application I/O there is no motivation to do so naturally and neither Eylon nor Christiansen provide any such motivation.

- Limitation 1.b.v is not taught by Eylon or Christiansen because no definition of the term "data stream" and subsequently "complete data stream" corresponds to any of the various meanings of the term that I was able to identify. The Board decided that the term "complete data stream" and "file" were synonymous. I have explained why I disagree with this construction of the term, identified existing use for the term "data stream" in 2008, and suggested what is a reasonable construction for this term. Regardless, none of the different meanings I could identify for this term would imply that a "complete data stream" is equiv-

alent to a file; accordingly, the data stream and complete data stream concepts of the '808 Patent are not taught by the cited prior art.

- Limitation 1.c.iv is not taught by the combination of Eylon and Christiansen. First, neither Eylon nor Christiansen teach the handling of both application I/O and paging I/O. I explain why application I/O and paging I/O are distinct. The '808 Patent motivates the decision to include the added complexity of supporting both application level I/O as well as paging I/O because it is more efficient for at least some classes of streaming application delivery. In addition, I note that the motivation to combine Eylon with Christiansen is defective: Eylon teaches implementing a file system, while Christiansen teaches implementing a file system *mini-filter* instead of implementing a file system *filter*. A file system filter is not equivalent to a file system driver. There is no analysis before the Board explaining why the implementation of Eylon as a Christiansen-style file system mini-filter is motivated. Further, I have provided descriptions of five distinct ways in which the system of Eylon could have been implemented as something other than a file system filter or file system mini-filter and are motivated to be chosen because they offer either simpler implementation with lower business risk or greater control over the implementation than is offered by a file system filter or file system mini-filter example.

- Limitation 1.c.v is not taught by the combination of Eylon and Christiansen. The cited prior art fails to capture the distinct identification of relevant application meaningful data ("assets" or "data packs"), the clear identification of these assets, the observation that such assets are duplicated and thus do not require re-transmission *even when their placement is not block aligned* and the ordering of these assets is dynamically reordered is substantively different than the static fixed-size block division of the cited prior art.

- Limitation 1.d is not taught in the prior art because the prior art teaches handling paging I/O, which is only initiated by the operating system, with application I/O, which is the only

type of I/O that an application can initiate — though components within the system may also initiate application I/O. Paging I/O is always non-cached. Application I/O is usually cached, but may be non-cached. A virtual file system *must* handle paging I/O but usually does not directly handle application I/O by using existing operating system functions for handling application I/O requests against a file system data cache. A file system filter or file system mini-filter may handle just application I/O, just paging I/O or both. For the system of Eylon implemented using the mini-filter model of Christiansen, a POSITA would understand that such a filter is most easily implemented to only handle non-cached I/O and to allow cached I/O to be satisfied using the normal mechanism against the file system data cache. A POSITA would appreciate that there is no benefit to handling both cached and non-cached I/O for the file and thus would only handle non-cached I/O. For the system claimed and taught by the '808 Patent, a POSITA would understand that such a mini-filter needs to satisfy the application I/O requests — which cannot be assumed to fall on page boundaries or be integral pages in length — to properly identify the relevant identifier of the "data pack" needed to satisfy the request. Similarly, a POSITA would understand that the mini-filter of the '808 Patent must handling paging I/O as a matter of correctness, to permit memory mapped file access. Finally, a POSITA would understand that the mini-filter of the '808 Patent must integrate with the Windows file system data cache to ensure that it is properly purged in cases where the cached page includes file regions that have not yet been streamed to the local system. Thus, this limitation of the '808 Patent is not taught by the cited prior art.

Thus, it would seem that while there are superficial similarities between the teachings of Eylon and the claims of the '808 Patent, there are also critical differences. Similarly, there are similarities between the teachings of Christiansen and the claims of the '808 Patent. In addition, there is a lack of motivation to combine Eylon with Christiansen. Indeed, the system of Eylon could be realized in 2008 using a variety of mechanisms, each of which is simpler than the proposed obvious combination of the Eylon-filter converted a a Christiansen mini-filter.

4. Independent Claim 14 I refer to my analysis for Independent Claim 1, as the language of this claim is substantially similar, with the additional limitation in this claim that it is installed on an existing system, such as might be done on first install, or during a software upgrade.

5. Dependent Claims 2-12 and 15-20 As discussed above the Petitioner has failed to demonstrate that the prior art teaches the limitations of the '808 patent. Since these dependent claims incorporate all the limitations of either Claim 1 or Claim 14, the Petitioner

6. Dependent Claim 13

D. Alleged Obviousness over Christansen, Eylon, and Pudipeddi: Claim 2

1. Overview of Pudipeddi (Ex. 1007) Pudipeddi looks to be the original Microsoft patent on their file system filter manager. It teaches the construction of a legacy file system filter driver (known as "filter manager" within the domain,) how the combination of legacy file system filters, filter manager, and file system mini-filters coexist. It demonstrates that the natural combination of a file system filter with a file system driver is to layer the file system filter (or mini-filter) on top of the file system driver.

2. Differences Between the Claimed Subject Matter and the Prior Art The claimed subject matter is a file system minifilter that implements a mechanism of streaming delivery of application defined data packs from a source location to the local computer system, such that the application can read the data that it needs to perform its assigned task.

The prior art consists of a file system driver that implements a mechanism of streaming delivery of uniform sized data blocks from a streaming application server to the client computer on which the streaming application is running. The prior art also consists of a fiter manager that layers itself ("attach") on top of one or more existing file system drivers to permit file system minifilter drivers to implement their functionality and a file system minifilter driver that replicated data between the local computer system and a remote network file server via a network redirector.

The '808 Patent cites the mechanism of Vinson (Ex. 2067), a virtual drive based solution like the solution of Eylon: "An index contains the information used by a client-side file system driver (FSD) to compute which chunk to download at runtime as it is needed. The FSD also creates a virtual drive, complete with drive letter, and places each target program that is to be accessed under a top-level directory on that virtual drive. Access to a chunk of a remote target program that has not been downloaded to the client machine may be granted or not depending on the type of access being requested (i.e. which volume or file function is being executed), an ID of a process attempting access, and optionally the path specified in the request for those requests containing paths." (Ex. 1001, Col 1, Lines 56-67) Thus, the '808 Patent motivates its own system by observing: "What is needed is a system and method that allows streaming delivery of program components from a source such as a networked server or tangible medium without creating a virtual drive or file system driver. The invention is directed to overcoming one or more of the problems and solving one or more of the needs as set forth above." (Ex. 1001, Col 2, Lines 32-37)

The Petitioner does not cite to any other reference that teaches the preference of a file system minifilter over a virtual drive. The Petitioner also fails to mention that their own streaming app delivery system, which was available from Microsoft since 2006 used the same virtual file system drive model, a fact that I know because it used the OSR File Systems Development Kit (FSDK) and I provided support to Microsoft as a client of OSR. (Ex. 2083).

Thus, as obvious as it might have been to combine Eylon with Christiansen and Pudipeddi in the manner the Petitioner and Dr. Houh have proposed, it was not sufficiently obvious that Microsoft chose to implement it in that way, despite having all the technological pieces necessary to do so.

3. Alleged Reasons for Combining the Teaching of the References As I have previously noted (e.g., in discussing Limitation 1.c.iv) the combination of Eylon with Christiansen taught by Christiansen is to layer Christiansen over Eylon as is normal with a file system minifilter over a file system drive. Since this does not achieve the Petitioner's goals, Dr. Houh instead creates a novel minifilter that implements the functionality of Eylon. In doing this, Dr. Houh and

the Petitioner fail to address the conflicts between the teachings of Eylon and the teachings of Christiansen. Now, the Petitioner has decided to also add Pudipeddi into the mix.

Pudipeddi describes the file system legacy filter that provides the support for minifilter drivers. Pudipeddi teaches (Ex. 1007, Fig. 2) that the way in which a filter and minifilter interact with a file system is by layering on top of the file system. This is also the teaching of Christiansen. Thus, according to these cited prior art references the normal combination of Pudipeddi is for Pudipeddi to attach to Eylon's file system(s), and then for Christiansen's minifilter to register with the filter manager (Ex. 1007, Fig. 2 and 3).

Unfortunately, this combination does not provide the benefits sought by the Petitioner, since it does not convert the file system driver of Eylon into a file system filter. Without that conversion, they cannot teach the claims of the '808 Patent.

Thus, the Petitioner proposes a novel transformation: to take the file system of Eylon, combine it with the file system filter of Pudippedi and the minifilter of Christiansen and produce a novel new file system minifilter which they argue makes the Claims of the '808 Patent obvious.

This obviousness analysis misses the step of justifying the conversion of Eylon into the legacy filter of Pudipeddi. It fails to address the conflics that arise between Eylon and Christiansen: Eylon requires an intelligent streaming application server, Christiansen relies upon a pre-existing network file system (or "redirector"). Does Dr. Houh's combined system abandon Eylon's teaching that the server pushes data to the client (Ex. 1005, Col. 6, Lines 59-62)? Does Dr. Houh's combined system implement the "no write back to the server" model that Eylon teaches (Ex. 1005, Col. 14, Lines 3-13) or the "write back to the server" model that Christiansen teaches (Ex. 1006, 2)? While Dr. Houh might simply appeal to the remarkable skill set of his POSITA, as an expert I do not see any obvious "right answer" to these questions.

Ironically, Pudipeddi teaches us that the motivation for filter manager is because: "Although this existing filter driver model provides a number of benefits including being highly flexible, there are also a number of inherent problems with it. For one, writing a file system filter is a non-trivial task, primarily as a result of the underlying complexity of a file system. Filters are highly complex pieces of software that are traditionally hard to debug and maintain. Much of the complexity

arises from the filters' handling of the packets, e.g., the need to understand and manipulate IRPs. As a result, reliability suffers, and at least one study has shown that filters have been responsible for a significant percentage of system crashes." (Ex. 1007, Col 1, Lines 55-65) I find this ironic because, from the perspective of an experienced Windows file systems developer, the interpretation and decoding of IRPs is one of the less taxing tasks one is asked to perform. Instead, the biggest complexity is dealing with the virtual memory system. This perspective is reflected in Nagar's book on file systems development (Ex. 2017pp. 194-356) as handling the file system data cache and dealing with demand paging make up $\frac{1}{3}$ of the book's content.

Thus, it is insightful to understand that the challenges Eylon overcame to implement a file system driver were not the challenges addressed by Microsoft's own filter manager. It also explains why the challenges I explained (see earlier discussion regarding Limitation 1.c.iv) that would arise are not addressed by the filter manager.

4. Conclusion The proposed combination defies the teachings of the references themselves because the normal combination of the file system of Eylon with the legacy filter of Pudipeddi and the minifilter of Christiansen would layer Pudippedi's driver on top of Eylon's file system, with Christiansen's minifilter. As I have previously explained (see discussion regarding Limitation 1.c.iv) the novel combination proposed by the Petitioner is flawed and considerably more difficult to implement than the Petitioner has presented.

The addition of Pudipeddi to Eylon and Christiansen does nothing to overcome these earlier defects. Thus, the combination of Eylon, Christiansen, and Pudipeddi are not obious.

E. Alleged Obviousness over Christiansen, Eylon, and Prahlad

1. Overview of Prahlad I have no information or opinion that is contrary to the Board's overview of Prahlad.

I do differ in my understanding of the teachings of Prahlad, however. The system of Prahlad teaches the construction of a change log that consists of a linear sequence of entries, each of which

describes a given change. This is a reasonable format for capturing log state and is like functionality that existed prior to Prahlad (the NTFS USN Journal).

I note that:

- The entries are variable size

- The entries are sequentially recorded in a flat file structure

- The entries have a temporal ordering, permitting replay needed for replication

- The entries correspond to changes to the local storage

What is missing from Prahlad to apply it in the way that the Petitioner proposes doing so is an efficient way of scanning such a log to find such markings. The Petitioner's analysis ignores the reality that a sequential list of variable size records indicating changes cannot be practically used to determine if a given region of the file has been delivered because such a determination would either require scanning the list each time a determination must be made or constructing an in-memory representation of the regions of the file that have been stored within the virtual file system. In the latter case, there is no benefit to using the teachings of Prahalad. In the former case a POSITA would know that linear scanning a potentially long change list is not a viable solution. Thus, a POSITA would not combine the teachings of Prahlad with the teachings of Eylon and/or Christiansen.

As I previously described in my analysis of Limitation 1.c.iv, Dr. Houh's proposed minifilter can be implemented without having any handling of cached I/O. However, by combining it with Prahlad, the resulting system must address several issues, including: (1) it **must** handle both types of I/O to ensure that the application I/O is captured, as is claimed by the '808 Patent; (2) it must decide how it will avoid duplicating entries in its list, since the ordinary application I/O will show up as cached I/O and againt as paging I/O. It is not possible to only log non-paging I/O because then the combined system will not work properly with memory mapped files, the modifications of which are not done using the read and write calls and thus file changes would be

missed. Thus, the choice confronting the Petitioner in their proposed combination is that they do duplicate work or they have incorrect data.

This remains a disadvantage over the teachings of the '808 Patent, which does not have these disadvantages as it does not teach the use of a linear scanned change log as taught by Prahlad.

2. Diferences Between the Claimed Subject Matter and the Prior Art Virtual file systems, by their very nature, keep track of what data is stored locally. Many different file systems, including the NTFS file system, which has been shipped by Microsoft as part of Windows since 1993, support the concept of a "sparse file". Such files have a special way in which they can mark that a region of a file has no locally stored data. Thus, the system of Eylon, being based upon a virtual file system, would have already had the ability to store specific information about the status of data that has been recorded locally, since that data is either present on the local system or not present on the local system.

3. Alleged Reasons for Combining the Teachings of the References As I have previously noted (analysis in Limitation 1.c.iv) the Petitioner has not provided any motivation to implement the system of Eylon as a file system filter. Accordingly, there is no reason to combine Eylon with Christiansen because the benefits of combining with Christiansen are those that arise simply because the Petitioner chooses to implement Eylon as a file system filter driver, rather than a file system driver as taught by Eylon.

In fact, the Petitioner's own argument here provides a further reason why the system of Eylon should not be implemented as a file system filter driver: Eylon's system already had a mechanism for marking data that had been received locally; this mechanism is lost when Eylon is converted from a virtual file system to a file system filter driver. Thus, to cure the defect they introduced in the Eylon-as-filter driver they choose to combine it with Christiansen — converting their filter in to a mini-filter, and then cure the defect they introduced by combining it with the sequential linear log record system of Prahlad.

The resulting system is complex, grossly inefficient, and unlikely to be usable. Thus, it is my opinion that the proposed combination of Eylon, Christiansen, and Prahlad is actually counter-

motivated by the unusable system that has been proposed as being the obvious combination here. To a POSITA the system of Eylon, Christiansen, and Prahlad is clearly inferior to the implementation of Eylon as a virtual file system.

In addition, I have previously described multiple alternatives to implementing Eylon as a file system filter driver. None of these other techniques would logically be combined with either Christiansen or Prahlad as neither of them solve any issues or limitations of these alternative implementation models.

In addition, the teachings of Eylon and Christiansen with respect write operations are in conflict; to combine these two teachings would require adopting the mechanism of one or the other and *neither* of Eylon nor Christiansen teach the limitations of Claim 13 of the '808 Patent.

Specifically, Eylon teaches that the client does not support modifications to the VFS: "As a result, operating system level operations such as copy or erase/delete files, etc., cannot be performed from the client side on the Server file system, further securing the streamed application from unauthorized access and use." (Ex. 1005,Col. 14, Lines 10-13) Eylon does not use the word "write" in the claims, specifications or diagrams. Eylon teaches: "It should be noted that in this overall system, there is preferably no direct access from the operating system level to the server 12. Communication with the server 12 is performed using a streaming data protocol implemented by, e.g., communication driver 170, and is generally limited in scope to the initiation of a streaming application and the subsequent processing of streamlets and forwarding of usage data." (Ex. 1005,Col 14, Lines 3-10.)

Christiansen teaches the handling of write I/O operations: "At block 815, a determination is made as to whether the create operation is attempting to open a remote object for write access. If so, processing branches to block 830; otherwise, processing branches to block 820. If the create operation is attempting to modify a remote object (the yes branch), the filter passes the operation to the redirector so that the remote content is updated. This allows the original content, not the content that is cached locally, to be updated." (Ex. 10062)

Finally, the combination of Eylon, Christiansen, and Prahlad ignores the claims and teachings of the '808 Patent that data packs may already be present on the local system (Ex. 1001, Col.

9). Thus, even with this combination, the system produced is still inferior to the system claimed and taught by the '808 Patent as it would require making duplicate copies of data that is already stored locally on alternative storage devices.

4. Conclusion The system taught by Prahlad is sufficient for replication because such changes need only be utilized one. The system of Prahlad is not viable for keeping track of the state of file regions as it is taught by Prahlad. Thus, the combination of Prahlad with Eylon and Christiansen, while possible, yields a system that will not be usable.

The virtual file system taught by Eylon already has a mechanism by which it tracks the regions of the file that have already been stored locally. To motivate the combination of Eylon and Christiansen with Prahlad, the Petitioner argues that this Eylon-as-minifilter no longer has the ability to keep track of the data that it has stored locally and thus must be augmented by the system of Prahlad to overcome this deficit. The resulting system is either unusable — because it is doing a linear scan of this growing log file of Prahlad — or it implements a tracking mechanism for the data that has been downloaded, which must have already existed in the embodiment of Eylon as a virtual file system.

Thus, a POSITA would reasonably conclude that there is no reason to combine these references.

F. Alleged Obviousness over Christiansen, Eylon, and Thind: Claim 13

As I have previously noted, Dr. Houh's analysis is flawed because he implicitly assumes that the best implementation of the teachings of Eylon is as a file system mini-filter driver. I disagree with this teaching: Eylon was well aware of Windows file systems and thus would have been familiar with Windows file system filter drivers. He chose to implement his system as a virtual file system because it was the simplest way to implement this using the available file system technologies.

The '808 Patent teaches us that a streaming on-demand application delivery system can be implemented as a file system mini-filter driver. Dr. Houh does not cite to any prior work that bridges the gap between the VFS of Eylon and the file system mini-filter of the '808 Patent. Dr.

Houh neither claims nor cites any evidence as to why the only option for implementing the VFS of Eylon is as a file system filter driver. Without the motivation to convert Eylon's VFS to a file system filter, the rationale that Dr. Houh adopted from Microsoft as to the benefits of a file system *mini-filter* over a file system *legacy filter* are not applicable.

Beyond this, I have described multiple other ways in which the system of Eylon could have been realized on Windows prior to 2008. None of these require a legacy file system driver and all were less risky than the system of Dr. Houh.

Thus, a POSITA would have no reason to combine the teachings of Eylon with Christiansen or Thind, since neither of the latter two apply to virtual file systems.

Finally, I would note that the system claimed and taught by the '808 Patent requires the ability to process cached I/O requests, which are typically initiated by an application as well as the requirement to process non-cached I/O requests. Most non-cached I/O requests are paging I/O requests and a paging I/O request can *only* be initiated by an operating system component. Applications running on Windows cannot directly invoke a paging I/O operation. The combination of Eylon with Christiansen and Thind is satisfied simply by handling paging I/O. There is no motivation provided to handle application I/O. In fact, handling both paging I/O and application I/O is more complex and thus there is a motivation for an embodiment of Eylon, Christiansen, and Thind *not* to handle application I/O operations.

It is my opinion that this would have been known to the Petitioner's POSITA and thus they would not have combined Eylon, Christiansen, and Thind to achieve the benefits of the '808 Patent's claims and teachings.

1. Overview of Thind (Ex. 1008) I have no information or opinion that is contrary to the Board's overview of Thind.

2. Differences Between the Claimed Subject Matter and the Prior Art The teachings of Eylon related to the implementation of a virtual file system on a platform, such as Windows. Windows file systems, since at least Windows NT 1993, have had multiple ways in which to track deletion state. The file system has no "issues" to solve here because it controls its

own data storage. However, there is a known issue with a file system filter driver — including a file system mini-filter driver — to detect that a file has been deleted.

This is because while the file system can properly serialize creation and deletion events, a file system filter driver cannot do so. I have taught about the complex issues of file deletion in file system filter drivers on Windows for more than two decades. Those problems do not exist for file systems, such as the virtual file system of Eylon.

Thus, the Petitioner introduces the mechanism of Thind to overcome a defect they introduced when they converted Eylon from a file system to a file system filter and then combined it with Christiansen to implement a file system mini-filter.

The prior art of Thind clearly states: "What is needed is a method and system for maintaining namespace consistency between selected objects maintained by a file system and a filter associated therewith." (Ex. 1008, Col. 1, Lines 45-47)

Thus, Thind is not required for the original system of Eylon because Eylon teaches us to build a virtual file systen on Windows — not a file system filter.

3. Alleged Reasons for Combining the Teachings of the References I have previously explained why there is no reason to combine Eylon with Christiansen. The implementation of Eylon as a virtual file system does not require the teachings of Thind to overcome the deficiency introduced when the Petitioner combined Eylon with Christiansen.

Thus, a POSITA, reviewing the work of Thind would not only not seek to combine it with Eylon and Christiansen, they would also view it as a reason not to combine Eylon with Christiansen, as it leads to this novel issue that requires the remediation taught by Thind.

4. Conclusion It is my opinion that the implementation of Eylon as a file system filter driver instead of the virtual file system that is taught by Eylon is not motivated by any of the evidence provided by the Petitioner. It is my opinion that the obvious combination of Christiansen (a file system minifilter) with Eylon (a file system) was to have Christiansen's minifilter attach to Eylon's file system. This is not the combination Dr. Houh proposes; he does not explain why his

combination is obvious in light of the teachings of Christiansen that layering is the natural way of combining file systems and filter drivers.

It is my opinion that the implementation of Eylon as a virtual file system was a reasonable solution to construct an embodiment of Eylon that would have been apparent to the Petitioner's POSITA, and that implementing Eylon as a file system mini-filter introduces additional complexity: the need to control the file system data cache, the need for an alternative allocation management strategy — as taught by the Petitioner's arguments to use Prahlad, and the inability to implement the proprietary data formats, encryption, and sparseness that were features of Eylon's original system. The complexity introduced by Dr. Houh's minifilter combining Eylon and Christiansen, though somewhat mitigated by removing features that Eylon teaches are important makes it clear that the POSITA would not choose to implement the combination of Eylon and Christiansen in the way that Dr. Houh proposes. Instead, the POSITA would either choose to implement the system as taught by Eylon or considered one of the several less technically challenging alternatives that I described.

III. Supplemental Information

In this section, I provide a more thorough explanation of various findings that I present in this second declaration; my goal in doing so is to provide extrinsic support for the various opinions that I express in this declaration.

A. Computer Science Undergraduate Education

Dr. Houh states that a person of ordinary skill in the art (POSITA or "Skilled Artisan") would have been "familiar with technologies known at the time, such as streaming technology, operating system methods for handling I/O requests, and loadable device driver management architectures such as filter manager and minifilter architecture." (Ex. 1003, p. 17).

My own investigation of the evidence for the relevant period does not support this contention in the context of undergraduate computer science education. Undergraduate CS education does ordinarily require students take coursework in operating systems. According to the ACM/IEEE Interim Computer Science Report in 2008 (Ex. 2068) operating systems core instruction consists of 20 instructional hours out of a total of 280 hours of CS instruction. Thus, operating systems education consists of approximately 7% of the total instructional time expected from an undergraduate student.

I further note that the 2008 report states that neither device driver architectures nor file systems are part of this core operating systems instruction. There is no mention of education about streaming technologies, operating system methods for handling I/O requests, loadable device driver management architectures, filter manager, and/or minifilter architectures.

Thus, the extrinsic evidence from the relevant time period suggests Dr. Houh's belief that this was the norm is not grounded in the learning from an undergraduate education.

B. Computer Science Graduate Education

Dr. Houh states that a person of ordinary skill in the art (POSITA or "Skilled Artisan") would have been "familiar with technologies known at the time, such as streaming technology, operating system methods for handling I/O requests, and loadable device driver management architectures such as filter manager and minifilter architecture." (Ex. 1003, p. 17). In § A. I observed that the ACM/IEEE report from 2008 did not indicate anything that suggested such material would be part of a normal undergraduate education.

To investigate graduate educational activities, I searched for and found syllabi from multiple graduate computer science institutions:

- University of Wisconsin, CS537; this is a graduate level operating systems course taught by Andrea Arpaci-Duseau, who is still a professor there. She is also co-author of an operating systems book, written after this time period, "Three Easy Pieces". There is some discussion of file systems. That information is consistent with my contentions: it is applications that define the structure of data within files and that file systems ignore that structure as part of providing a uniform abstraction over a block-structured media device. There is no discussion of file system filters, file system mini-filters, or Windows (Ex. 2069).

- Massachusetts Institute of Technology, 6.828 Operating Systems Engineering; this is a graduate level operating systems course, taught by Frans Kaashoek, who was a professor in at least 1993 and remains there today. This includes the time period when Dr. Houh received his PhD; it is possible Dr. Houh took Dr. Kaashoek's graduate operating systems course. The 2006 syllabus is available online still [1]. This course has one lecture on File Systems, and a second on High-performance File Systems. There is no mention of device drivers, loadable driver architectures, file systems filters, file system mini-filters, or Windows (Ex. 2070, Ex. 2071, Ex. 2072, Ex. 2073, 2074, and 2075).

[1] https://dspace.mit.edu/handle/1721.1/92292

- University of Manitoba, 074.784 Operating Systems Design and Implementation; this is a graduate level course in operating systems design and implementation, taught by Peter Graham. Dr. Graham worked at the University of Manitoba from 1996 to 2021 when he retired. The 2007 Syllabus is available online (Ex. 2076). His course had coverage of device drivers and file systems. There is no mention of loadable driver architectures, file system filter drivers, file system mini-filters or Windows.

Thus, the only information that I could find suggested that these topics were not ordinarily taught to graduate students and thus would not normally be known to a POSITA outside the context of work experience.

C. Computer Science Textbooks

Dr. Houh states that a person of ordinary skill in the art (POSITA or "Skilled Artisan") would have been "familiar with technologies known at the time, such as streaming technology, operating system methods for handling I/O requests, and loadable device driver management architectures such as filter manager and minifilter architecture." (Ex. 1003, p. 17). As I have noted in § A. and § B. this understanding is not something they would have ordinarily gained in an undergraduate or graduate educational context.

I continued my evaluation of this by reviewing computer science textbooks from this era, both because someone particularly interested in this topic might choose to learn from such a textbook and also because it demonstrates what one might find in a broader range of instructional settings than those I have previously described.

My search identified the following textbooks, related to operating systems and of which file systems are one aspect:

Operating Systems Design and Implementation — the third edition of this book was published in January 2006, authored by Dr. Andrew Tanenbaum. This book contains an entire Chapter about File Systems. It does not mention loadable device driver management architectures such as filter manager and minifilter architecture. (Ex. 2077)

Operating Systems: Three Easy Pieces — this book was first published in 2008 and continues to be updated and made generally available. The authors are Andrea Arpaci-Dusseau and Remzi Arpaci-Dusseau. The book contains extensive descriptions of both device drivers and file systems. There is no mention of loadable device driver management architectures such as filter manager and minifilter architecture. There is an extensive discussion about the details and considerations about file system design. (Ex. 2078)

Operating System Concepts — this book remains a commonly used textbook. The seventh edition was published in 2005. The authors are Abraham Silberschatz, a professor of CS at Yale, Greg Gagne, Chair of the CS and Math Division at Westminster College, and Peter Galvin, a CTO of Corporate Technologies. It has two chapters on file systems. The book does reference "dynamic loading" but it is in the context of application programs, not device driver management architectures. There is a discussion of Windows, but it does not cover the Windows operating system, including filter drivers, though it does cover a feature of the NTFS file system, namely access control lists ("ACLs"). (Ex. 2079)

Thus, educational texts do not provide the grounding necessary to understand the concepts that Dr. Houh ascribes to a POSITA/Skilled Artisan. It is my opinion, based upon my personal experience in this field within the relevant time frame that no one learned about Windows file systems, device drivers, loadable driver architectures or file system mini-filters in an undergraduate or graduate education program, or from self-study of operating systems textbooks.

D. VFS versus IFS

The term VFS is one of art in the UNIX community: the term VFS was introduced by SUN Microsystems. This technology was incorporated into UNIX and released as part of UNIX SVR4 by AT&T in 1988 (Ex. 2080). This included two critical changes that are at the core of understanding both the work of Eylon and the '808 patent: the introduction of memory mapped file access, and the merge of the file system *buffer* cache with the operating system *page* cache. The best reference

for deeper insight here is *UNIX Filesystems: Evolution, Design, and Implementation*, which was published in 2003 (Ex. 2081.

The Linux operating system adopted the VFS model for its own file systems, and also was using a unified page cache by 2008. (Ex. 2085) In *Understanding the Linux Kernel, Third Edition* the authors provide considerable detailed information about how the VFS layer in Linux works, as well as how the page cache is used (Ex. 2082). I note that this is consistent with my understanding and clearly distinguishes application I/O, which is normally performed by copying data from the page cache, from paging I/O, which is done as a result of a page fault or dirty page write-back operation.

The UNIX and Linux operating systems have a long history of novel file systems development. The Windows operating system does not. Even today, there remains a single book about developing file systems for Windows (Ex. 2017). I do not know of any academic literature that describes developing file systems for Windows, while there are numerous such papers, with an entire conference (the File Systems and Storage Technology conference) that each year has multiple file systems papers.

Windows does not have a virtual file systems layer. This makes the adaptation of the vast depth of information about UNIX and Linux file systems difficult for anyone other than a domain expert to convert. Windows has an *installable file systems* layer. One might think these two are equivalent; they are not. I say this having developed a VFS file systems layer for the Windows kernel (Ex. 2083) and written a book about Windows kernel mode device driver development (Ex. 2084).

Knowledge of Windows file systems was, in 2008, and even today remains a rare skill set.

E. UNIX/Linux Page/Buffer Cache

Windows has always had a file system data cache In Figure 3 I have included a diagram that I first used prior to 2008 when teaching about the behavior of the Windows file system data cache

and its interaction with Windows file systems. I used an earlier version of this same diagram prior to 1998, as well.

UNIX and Linux were originally implemented with a "buffer cache" but not a file system data cache. The file system data cache of UNIX was introduced in UNIX System V Revision 4: "SVR4 implemented what is commonly called the page cache through which all file data is read and written. This is actually a somewhat vague term because the page cache differs substantially from the fixed size caches of the buffer cache, DNLC, and other types of caches." (2080, page 146). This distinction is further described as: "Prior to SVR4, all I/O went through the buffer cache. Each buffer pointed to a kernel virtual address where the data could be transferred to and from. With the change to a page-based model for file I/O in SVR4, the filesystem deals with pages for file data I/O and may wish to perform I/O to more than one page at a time." (2080, page 153). Note that UNIX System V Release 4.0 was released in 1989, well before the time of Eylon.

The Linux buffer cache was replaced prior to 2008 with a page cache comparable to that present in UNIX System V Release 4: The "page cache" of Linux is equivalent to the page cache of UNIX and the file system data cache of Windows: "The page cache is the main disk cache used by the Linux kernel. In most cases, the kernel refers to the page cache when reading from or writing to disk. New pages are added to the page cache to satisfy User Mode processes's read requests. If the page is not already in the cache, a new entry is added to the cache and filled with the data read from the disk. If there is enough free memory, the page is kept in the cache for an indefinite period of time and can then be reused by other processes without accessing the disk." (2082). This change occurred prior to 2006 and thus was known prior to the '808 patent.

In each of these operating systems, there is a distinct process outside the control of the ordinary application that implements a *write back* policy for cached data. In Windows file system cached data that contains modifications is written back by two specific kernel threads: the *Modified Page Writer* and the *Lazy Writer*, as shown in Figure 3. The Modified Page Writer implements logic inside the virtual memory manager component of the Windows kernel; it has a policy that is driven by ensuring the amount of modified ("dirty") data in mapped files remains below certain critical thresholds that affect virtual memory behavior. The Lazy Writer implements logic inside the

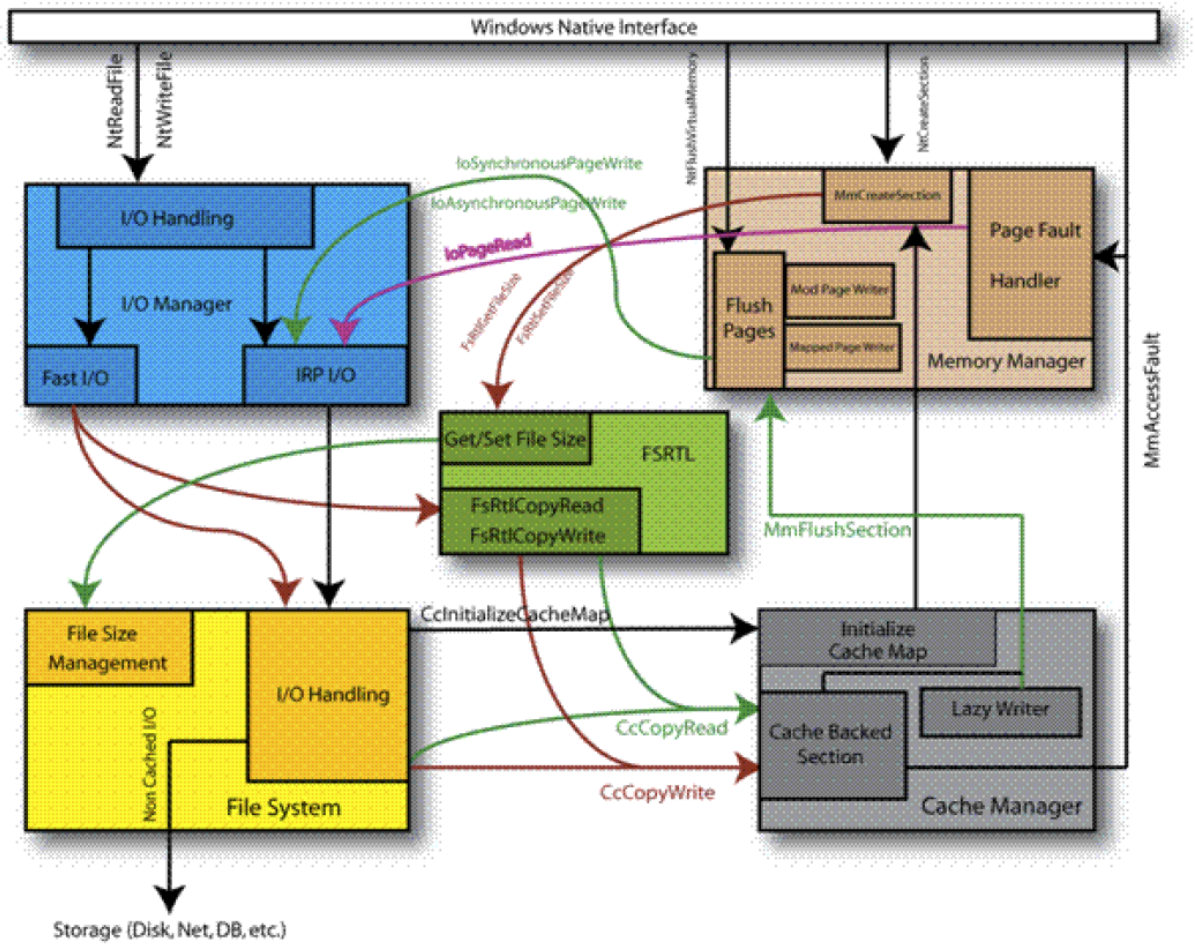

Figure 3: Windows File Systems/VM Interactions (2009)

Windows Cache Manager to ensure that cached file system data is written back to storage within some window. UNIX and Linux have a similar policy mechanism for writing modified ("dirty") data back to the underlying storage.

F. Streaming Technology

The term "streaming technology" is quite broad. Google acquired YouTube in 2006 [2], so certainly streaming technology was available. In addition, Netflix streaming vide delivery was announced in January 2007 [3]. Thus, I agree that streaming technologies were well known, but it

[2] https://www.businessofbusiness.com/articles/a-brief-history-of-video-streaming-by-the-numbers/
[3] https://www.nytimes.com/2007/01/16/technology/16netflix.html

is a substantial leap to conclude that these common examples related using streaming techniques to delivering data content. The information that I found reinforced my own recollection that this was considered to be primarily an internet technology, such as might be used inside a web browser [4] [5]. Streaming data was also used to describe the collection and processing of data from remote sensor devices (Ex. 2097), though I would not argue that was the intent for the '808 Patent or any of the cited prior art.

G. UNIX/Linux File Systems

UNIX-like operating systems, which includes Linux, have an I/O model in which the application view of essentially everything is that of a byte addressable file. The *block* abstraction is one that is largely relegated to the core operating system itself, such as device drivers. The *page* is a concept that originates with how *virtual memory* is allocated: we normally use the term "page" to refer to the allocation unit used for describing physical memory. Modern processors utilize a table-driven mechanism for converting *virtual* addresses to *physical* addresses. The term "page" is itself ordinarily treated as abstract, because this size is usually a function of the computer processor itself. For example, the Intel 80386 and subsequent processors ordinarily use a 4KB page, though there are special modes that allow for larger pages of 2MB, 4MB, or 1GB as necessary. Windows on Intel's Itanium processor actually used an 8KB page size — the Itanium processor supported a range of page sizes and it was the responsibility of the operating system to choose the particular page size from one of the available sizes.

This was known to both Eylon and Christiansen, as they were experts in the field of computer storage and would be familiar with how virtual memory hardware worked because virtual memory systems and file systems are tightly linked together in managing data: memory provides fast access to the data that application files require but lack persistence while file systems provide a general abstract model of named objects typically backed by persistent storage. Thus, there is

[4] https://tech.ebu.ch/docs/techreview/trev_292-kozamernik.pdf
[5] https://www.courses.psu.edu/ist/ist250_fja100/ist250online/Content/T7L4_streaming.htm

considerable movement of information between the two: loading code and data into memory for use during run-time, storing data onto disk to persist it.

Memory also happens to be addressable at fine granularity: all modern CPUs use the byte as a basic unit of addressing. Some processors had coarser access granularity, but the operating system and the code generation tools would transparently implement the byte-addressable abstraction. For example, to address a single byte on a machine that can only access eight bytes at a time (such as the Digital Equipment Corporation "Alpha" processor, a platform on which UNIX, Linux, and Windows NT were supported for several years in the 1990s) the CPU would load the eight byte block containing the one byte desired, then using bit masking and shifting operations leave only the one byte value present in the CPU register, on which the operation would then be performed. A similar inverse operation would be done when updating the larger block. Thus, between the CPU, the operating system, and the compiler tools, applications running on UNIX, Linux, and Windows systems all have the same "byte addressable" abstraction model.

This idea of fetching a larger block, modifying a smaller part, and then storing the larger block back is not unique to processors and memory: it is the same technique that we see used in storage systems, where a single byte update within a file consists of loading the data from disk into memory, then using the byte addressable support provided by the CPU, operating system, and programming tools to update the correct byte. Subsequently, the entire block of data, complete with the one byte that has been updated, is written back to the storage media.

Thus, there is an important disconnect in the thinking processes of an application program, who is used to a byte-addressable paradigm for memory and file access, and an operating system component, who is used to supporting a block oriented paradigm for memory and storage access. Having said that, I note these distinctions are not rigid: an application program on UNIX, Linux, or Windows can explicitly request block oriented access, such as by using a special option that asks for "non-cached" or "direct" access. Similarly, an operating system component can exploit the byte-addressable nature of memory.

I observe these distinctions, however, because they directly bear on what the assumptions are of someone in these two respective fields: an application programmer (including those that write

business applications software and games) will think of everything as being byte addressable. A kernel programmer (including those that write device drivers and file system drivers) will think of everything as consisting of fixed sized allocation units.

H. Windows File Systems

A Windows file system, unlike a UNIX/Linux file system, is expected to integrate with a special caching layer in Windows: the "cache manager". This cache manager is the interface between the Window memory manager and the Windows file systems, and logically serves as the Windows mechanism for integrating the page cache and buffer cache. The distinction is that Windows NT never had a buffer cache, it only ever had a page cache.

One of the important functions of media file systems, such as Microsoft's NTFS file system, is to remove these media-imposed restrictions from being similarly imposed on application programs, provided those application programs do not request a special "direct to media" mode. Thus, an ordinary application could be written so that it was *not* cognizant of these media-imposed limitations.

Note that while I have used the limits for a "typical hard disk drive" of the period in question, there was actually no requirement that media support this specific size. We can see this by looking at the interfaces applications could use to query for this information.

For Windows, this was done using NtQueryInformationFile [6]:

```
__kernel_entry NTSYSCALLAPI NTSTATUS NtQueryVolumeInformationFile(
  HANDLE                 FileHandle,
  PIO_STATUS_BLOCK       IoStatusBlock,
  PVOID                  FsInformation,
  ULONG                  Length,
  FS_INFORMATION_CLASS FsInformationClass
```

[6] https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/ntifs/nf-ntifs-ntqueryvolumeinformationfile

```
);
```

This API has been available in Windows since at least Windows NT 3.1, which was released in 1993. It is a general purpose interface for obaining information about a particular file.

One specific type of this query is for the file size information [7]:

```
typedef struct _FILE_FS_SIZE_INFORMATION {
  LARGE_INTEGER TotalAllocationUnits;
  LARGE_INTEGER AvailableAllocationUnits;
  ULONG         SectorsPerAllocationUnit;
  ULONG         BytesPerSector;
} FILE_FS_SIZE_INFORMATION, *PFILE_FS_SIZE_INFORMATION;
```

I would note that the `BytesPerSector` field is that which corresponds with the smallest atomic unit of storage *for the device*. I note that separately, Windows describes how many of these sectors are allocated at a time in the "SectorsPerAllocationUnit" value returned in this call. The NTFS specific term for this is "cluster"; other file systems use different terminology. Thus, the logical unit of breaking up a file **from the storage system perspective** is going to be the allocation unit, since this would correspond to the unit of allocation within the file system itself and is thus likely to be the most efficient in terms of the storage system.

Dr. Probert, of Microsoft, provides a useful talk on the Windows "Cache Manager"; while presented in 2010, the information that he provided there is consistent with the information that I have been teaching since 1996. I have included a copy of the relevant slide in Figure 4.

The reason to include this picture is to point out that there are *two* paths through this system labeled *cached I/O* and *noncached I/O*. **All** paging I/O is issued as a page-sized non-cached I/O operation, regardless of the operating system. This is because the size of a page is a function of the virtual memory system.

[7] https://docs.microsoft.com/en-us/windows-hardware/drivers/ddi/ntddk/ns-ntddk-_file_fs_size_information

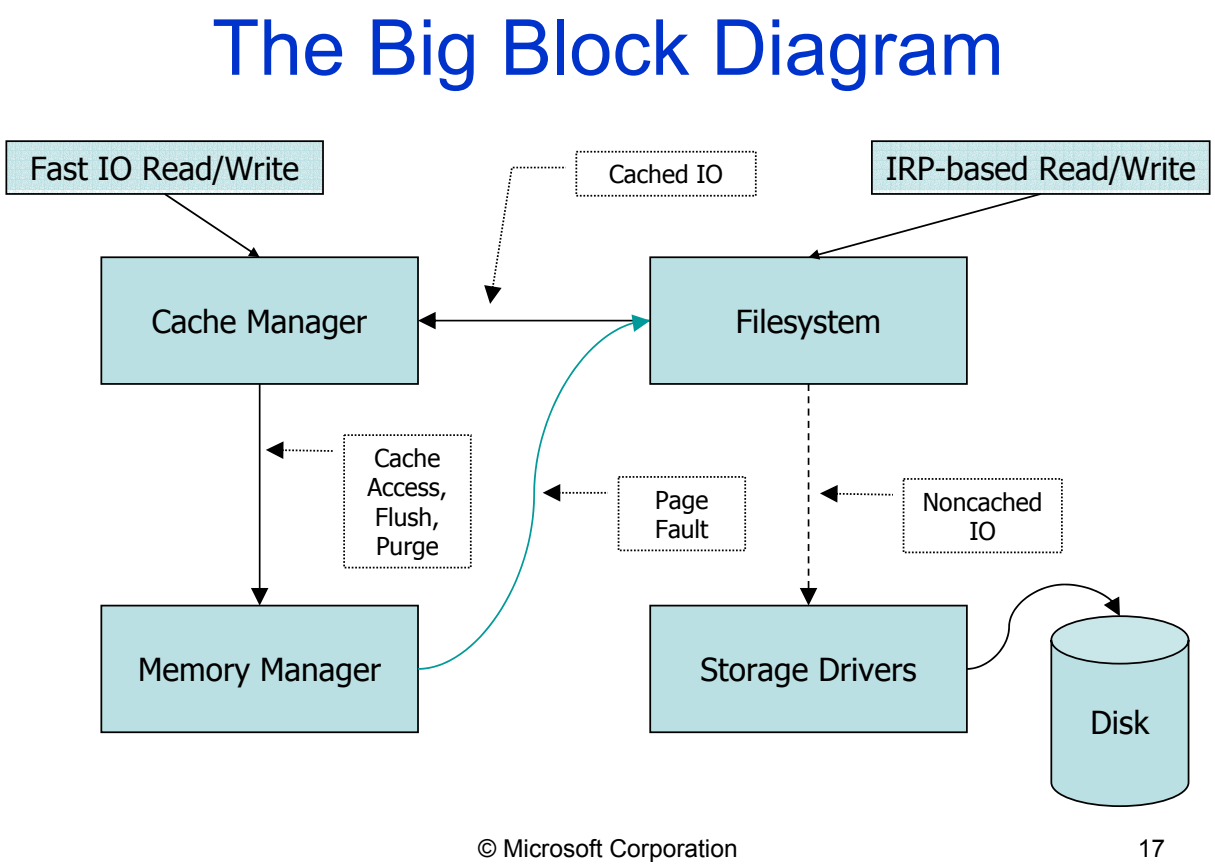

Figure 4: Dr. David Probert of Microsoft, *Windows Kernel Internals: Cache Manager*, 2010

I. Application versus Paging I/O

There is a misunderstanding as to the meaning of the '808 Patent's description of application I/O and Paging I/O: "'Patent Owner asserts that the '808 patent distinguishes between paging I/O operations and the "arbitrary offset and length I/O operations" associated with "data packs." Prelim. Resp. 23 (citing Ex. 1001, 7:50–55). But the patent treats a paging I/O operation as a type of read operation and does not distinguish "data packs" for a paging I/O operation from "data packs" for any other type of read operation. See Ex. 1001, 7:23–60, Fig. 4."' (Ex. Paper 8p. 22)

Eylon teaches the distinction between "paging I/O" and "data I/O": "Thus, as the streaming application executes and paging or data I/O requests are generated to retrieve require code or

data, the operating system will automatically direct it to the FSD driver which then passes it in the proper format to the VFS caching system for processing." (Ex. 1005,Col 4, Lines 17-22). Eylon also confirms this understanding of paging being tied to the size of a page on the computer platform: "For example, standard Windows systems utilize a 4 k code page when loading data blocks from disk or in response to paging requests." (Ex. 1005,Col. 3, Lines 63-65).

The '808 patent teaches that a "paging I/O" is distinguished from a "normal read": "a determination is made if the call is a paging I/O or a normal read. Paging (sometimes called swapping) is the transfer of pages between main memory and an auxiliary store, such as hard disk drive, which allows use of disk storage for data that does not fit into physical RAM." (Ex. 1001, Col 7, Lines 49-54). It also teaches that there is a special step required for paging I/O: "In a paging I/O, compressed packs are unpacked into allocated paging memory buffers at missing offsets in step 428, and control proceeds to step 432" (Ex. 1001, Col 7, Lines 54-56) versus "normal I/O": "In a normal read I/O, a new read request is created and sent for the same allocation to a lower filter driver and the original buffer is filled, as in step 438." (Ex. 1001, Col. 7, Lines 56-60). This distinction is also demonstrated in Figure 4.

Thus, both Eylon and the '808 patent teach the distinction between paging I/O and regular I/O. Because I have decades of teaching and explaining this difference, I am careful to explain the context of these operations and the limitations of them and I use the terminology that I have found to be clearest: application I/O and paging I/O. Only the operating system may issue paging I/O; such calls are not available to applications.

An *application* can only issue "normal I/O", which is what I refer to as "Application I/O". An application may request that the I/O operation be satisfied directly from the original data source, which is known as non-cached I/O. Ordinarily, however, an application does not specify non-cached I/O, in which case the read and write I/O operations are satisfied by an operating system managed data cache.

The *operating system* can issue both "normal I/O" and "paging I/O". When a virtual address is referenced by code running on the computer system and the virtual address is not currently defined, the computer system generates a *page fault*. Eylon teaches this: "Turning to FIG. 6A, when a

Page Fault exception occurs in the streaming application context, the operating system issues a paging request signaling that a specific page of code is needed for the streaming application." (Ex. 1005, Col 4, Lines 34-37). This is the mechanism by which an operating system can logically map the contents of files into a virtual address space, retrieving the correct data contents for the corresponding virtual address as needed.

Thus, for all mainstream operating systems, when an application issues a normal write I/O, the data is copied from the application's buffer into the file system data cache. This *can* trigger a read page fault if only a portion of the page is being overwritten by the application read I/O. Some time later, either the Mapped Page Writer or the Lazy Writer threads will write the dirty pages back to the file system, which will then store that data so it can be retrieved at a later time. The benefit of this approach are so profound that three independently developed operating systems have settled on the same technique for improving system performance.

I recall that when I first learned the details of Windows file systems and how they integrated with Virtual Memory (1994) I was rather surprised at how similar the file system data cache of Windows NT was to the page cache of SunOS/Solaris — the basis of the UNIX System V Release 4 page cache. Linux adopted the same page cache model prior to 2008: "First, even though this program uses regular read calls, three 4KB page frames are now in the page cache storing part of scene.dat. People are sometimes surprised by this, but all regular file I/O happens through the page cache. In x86 Linux, the kernel thinks of a file as a sequence of 4KB chunks. If you read a single byte from a file, the whole 4KB chunk containing the byte you asked for is read from disk and placed into the page cache. This makes sense because sustained disk throughput is pretty good and programs normally read more than just a few bytes from a file region. The page cache knows the position of each 4KB chunk within the file, depicted above as #0, #1, etc. Windows uses 256KB views analogous to pages in the Linux page cache." (Ex. ex:page-cache).

Paging I/O is different than application I/O in several key ways:

- Paging I/O is always done in page-size units on page-size boundaries. The file system may truncate a paging write that is larger than the end of file. The operating system must zero fill any portion of a page beyond the end of file.

- Paging I/O is always done non-cached. That is because the request to read or write the page must be done against the backing store, where the data is itself stored.

- Paging I/O is never directly initiated by an application program. (Ex. 2088 p. 2047, 2088 p. 2068, and 2066)

- Paging I/O is executed in "arbitrary thread context". What this means is that when a file system receives a paging I/O it *does not know* the context in which it is being executed. Accordingly, there is no simple way to map from the paging I/O back to the application that issued the paging I/O.

Application I/O can be cached or non-cached. Cached application I/O is the norm (default) on UNIX, Linux, *and* Windows, but both must be supported. Cached I/O operations have no restrictions on the alignment or size of their I/O operations. Non-cached I/O operations have documented alignment and size restrictions.

Applications issue I/O operations to their files to retrieve specific objects that have meaning to the given application. While the integrated file system data cache/page cache model of modern operating systems provides an efficient implementation it discards any knowledge of the access pattern of the application to the underlying data.

These distinctions are important when constructing file systems and file system mini-filter drivers. By default, a file system will ordinarily *not* handle cached I/O operations and will instead either have the OS handle it directly (e.g., via the FastIo path in Windows and the read_iter/write_iter path in Linux). Thus, what this means is that a typical file system on UNIX, Linux, *or* Windows will only handle non-cached I/O, which includes all paging I/O and specifically distinguished application I/O.

The primary source of paging I/O operations is to satisfy page faults from the file system data cache of Windows, or the page cache of UNIX and Linux. These caches are more thoroughly described in subsection E..

Documentation that explains paging I/O are available from Microsoft (Ex. 2088 p. 2047, 2066, 2088 p. 2068) and note these are clearly documented as internal Windows kernel functions only — Paging I/O is initiated by the OS, not by the application.

In addition, Microsoft documents Win32 APIs (Ex. 2091, 2090, 2089) as well as native Windows APIs for performing cached and non-cached I/O (which excludes paging I/O). (Ex. 2093, 2092, 2101, 2102). Note that this is a demonstrative list, not a definitive list.

J. Encryption

The use of encryption in data storage is an area in which I have worked for many years. It is this work that led me to create the "isolation" filter driver model that is necessary to properly construct a working embodiment of the '808 teachings.

Specific needs of storage encryption, particularly in end user devices of the type that are the subject of this patent can be found in the National Institute of Standards Guide (Ex. 2100)

K. Compression

Due to time limitations, I am simply referring to Wikipedia's description of data compression. Such compression is remarkably difficult to implement in a file system filter driver, based upon my real-world experience doing so. The NTFS compression mechanism is incompatible with NTFS encryption (for example) and uses a maximum compression region of 64KB. (Ex. 2103)

L. User Mode File Systems

User mode file system is a mechanism that existed prior to the time of the '808 Patent. Certainly the Petitioner's POSITA would have been aware of this work. For example, there were academic papers on *re-implementing* FUSE ("File Systems in User Space") in 2007 (Ex. 2098) Indeed, the original File Systems in User Space work was published in 1993 (Ex. 2099). Eylon might not have

chosen to implement his file system on Windows in this fashion because FUSE was not supported on Windows (though it is supported on Windows now).

M. File System Development Kit

The company that I partially owned and for which I worked doing file systems development offered the File Systems Development Kit (FSDK) prior to 1998 (Ex. 2083), and it was still commercially available in 2008; it is still listed as being available on the osr.com website (Ex. 2087).

The OSR File Systems Development Kit permitted developing file systems on Windows with considerably less effort than was required to develop an entire file systems solution from scratch. Using it would have been a potential alternative to implementing the system of Eylon as a file system filter driver.

N. RDBSS

The RDBSS model is the file system mini-redirector model that Microsoft has made available on Windows since Windows XP. This model, which is still documented today by Microsoft, describes how to construct a mini-file system driver that supports caching of remote data using a simplified implementation interface that integrates cleanly into the Windows platform.

Microsoft continues to document how to use this library to construct a file system mini-redirector (Ex. 2086).

Mini-redirector development was supported and well-documented by Microsoft. It represents an important potential design approach to constructing the system of Eylon.

IV. Exhibits

Exhibit #	Reference Name
2044	OSR Website (Seminars) 1997
2045	OSR Website (File Systems Seminar) 1997
2046	OSR Website (File Systems Seminar) 2007
2047	OSR Website (File System Mini-filter Seminar) 2007
2048	Windows NT IFS Frequently Asked Questions
2049	IoAllocateIrpEx
2050	IoBuildAsynchronousFsdRequest
2051	Wikipedia: File Systems Feature Comparison
2052	NTFS vs. FAT32: Which to Choose for Windows XP Install
2053	Wikipedia: Server Message Block
2054	US7,444,0111 B2 Lin, *et. al.* Truth on Client Persistent Caching
2055	US7,650,514 B2 Howell, *et. al.* Scalable Leases
2056	US8,121,061 B2 Rajaram, *et. al.* Granular Opportunistic Locking
2057	Implementing Name Provider Callbacks
2058	Callback File System
2059	Virtual Hard Disk format
2060	Windows Shell Programming Guide
2061	Mount ISO image on Windows Vista
2062	Wikipedia: File Explorer
2063	Architecture of the Kernel Mode Driver Framework
2064	Introduction to the WDF User Mode Driver Framework
2065	Developing File Systems for Windows (April 2005)
2066	IoAsynchronousPageWrite

Exhibit #	Reference Name
2067	US 6,453,334 B1 (Streaming Download with persistent caching)
2068	ACM/IEEE Interim Computer Science Curriculum Report 2008
2069	University of Wisconsin at Madison, CS 537 Operating Systems Syllabus
2070	Massachusetts Institute of Technology, Operating Systems
2071	Massachusetts Institute of Technology, File Systems
2072	Massachusetts Institute of Technology, High Performance File Systems
2073	Massachusetts Institute of Technology, Overview
2074	Massachusetts Institute of Technology, Naming
2075	Massachusetts Institute of Technology, Syllabus
2076	University of Manitoba Operating Systems Design and Implementation
2077	Operating Systems Design and Implementation, Third Edition
2078	Operating Systems: Three Easy Pieces
2079	Operating System Concepts, Seventh Edition
2080	Wikipedia: UNIX System V
2081	UNIX Filesystems: Evolution, Design, and Implementation (2003)
2082	Understanding the Linux Kernel
2083	OSR File Systems Development Kit
2084	Windows NT Device Driver Development
2085	Page Cache, the Affair Between Memory and Files
2086	The RDBSS Driver and Library
2087	OSR FSDK (2021)
2088	Windows IFS Guide
2089	Windows Documentation for Win32 ReadFileEx
2090	Windows Documentation for Win32 ReadFile
2091	Windows Documentation for Win32 OVERLAPPED structure
2092	Windows Documentation for Native NtWriteFile
2093	Windows Documentation for Native NtReadFile

Exhibit #	Reference Name
2094	Windows Documentation for File Buffering
2095	Microsoft Application Virtualization (App-V) for Terminal Services
2096	Wikipedia: Microsoft App-V
2097	Mining Data Streams: A Review
2098	ReFUSE: Userspace FUSE Reimplementation Using puffs
2099	File Systems in User Space
2100	Guide to Storage Encryption Technologies for End User Devices
2101	NtReadFileScatter Documentation
2102	NtWriteFileGather Documentation
2103	Wikipedia: Data Compression
2104	Wikipedia: EFS on Windows

Declaration

I, William Anthony Mason, do hereby declare and state that all statements made herein of my own knowledge are true and that all statements made on information and belief are believed to be true; and further that these statements were made with the knowledge that willful false statements and the like so made are punishable by fine or imprisonment, or both, under Section 1001 of Title 18 of the United States Code.

W Anthony Mason

Digitally signed by W Anthony Mason
Date: 2021.07.22 14:11:44 -07'00'

Dated: July 22, 2021

William Anthony Mason